STUDIO USER MANUAL · 사용자 매뉴얼

# NeoFlow

## NeoFlow Studio 사용자 매뉴얼

NeoFlow Studio · ETL Job 작성 · 동기화 · 배포 가이드

| 항목 | 내용 |
| --- | --- |
| 문서명 | NeoFlow Studio 사용자 매뉴얼 |
| 문서 버전 | v1.0.2 |
| 작성일 | 2026-10-07 |
| 제조사 | 주식회사 위블리스 |
| 대상 제품 | NeoFlow v1.0 — NeoFlow Studio (v1.0.0) |

© 2026 주식회사 위블리스

# 개정 이력

| 버전 | 일자 | 작성자 | 변경 내용 | 승인 |
|---|---|---|---|---|
| v1.0 | 2026-06-30 | 운영팀 | 최초 작성 | — |
| v1.0.1 | 2026-09-29 | 주식회사 위블리스 | 설치·제거 장 신설, NeoFlowContext·라이선스 메시지 정정 | — |
| v1.0.2 | 2026-10-07 | 주식회사 위블리스 | 제품 구성과 시험 범위·자체 개발 기능 목록 신설 | — |

# 목차

개정 이력 2
목차 3
1. 개요 4
1.1 제품 구성과 시험 범위 4
1.2 자체 개발 기능 목록 5
1.3 전체 작업 흐름 6
1.4 이 매뉴얼 보는 법 6
2. License 등록 7
2.1 라이선스 등록 방법 7
2.2 라이선스 저장 위치 7
2.3 자주 보는 메시지 8
3. NeoFlow Server 연결 정보 등록 9
3.1 API token 발급 (NeoFlow Server) 9
3.2 Preferences 페이지 열기 12
3.3 프로파일 관리 12
3.4 프로파일별 입력 항목 13
3.5 연결 테스트 13
3.6 저장 및 Job 으로의 주입 원리 14
4. NeoFlow 컴포넌트 16
4.1 tNeoFlowPre — BizDate 구간 가져오기 16
4.2 tNeoFlowPost — BizDate 워터마크 커밋 18
4.3 tNeoFlowThread — 서브잡 병렬 실행(시차 기동) 20
5. Job 파라미터 받기 (NeoFlowContext · NeoFlowJobApi) 22
5.1 파라미터가 Job 에 도달하는 경로 22
5.2 NeoFlowContext routine (기본 포함) 22
5.3 Job 에서 사용 22
5.4 동작 규칙과 제약 23
5.5 NeoFlowJobApi 로 서버 값 조회 24
6. Sync to NeoFlow Server 26
6.1 실행 26
6.2 마법사 단계 26
6.3 내부 동작 요약 29
6.4 Sync 전후의 Job 처리 30
7. Build Job (NeoFlow Deploy) 31
7.1 실행 31
7.2 주요 항목 31
7.3 배포 실행과 결과 32
8. 설치와 제거 33
8.1 시스템 요구 사항 33
8.2 압축 해제 33
8.3 JDK 지정 (NEO_BD-win-x86_64.ini) 33
8.4 첫 실행과 버전 확인 34
8.5 제거 35
9. 부록 36
9.1 빠른 점검 체크리스트 36
9.2 자주 묻는 질문 36

# 1. 개요

NeoFlow Studio 는 Eclipse 기반 오픈소스 ETL 도구인 Talend Open Studio 8.0.1 을 기반으로, NeoFlow 전용 기능을 추가한 ETL 개발 도구입니다. 사용자는 NeoFlow Studio 비주얼 환경에서 ETL Job 을 설계한 뒤, NeoFlow 전용 컴포넌트와 메뉴를 이용해 작성한 Job 을 NeoFlow 서버로 동기화·배포합니다.

이 매뉴얼은 당사가 자체 개발한 NeoFlow 전용 기능의 사용법을 다룹니다. Talend Open Studio 8.0.1 이 제공하는 일반 기능(표준 컴포넌트 사용법·메타데이터 등)은 Talend Open Studio 공식 문서를 참고하세요. 대상 제품은 NeoFlow v1.0 이며, NeoFlow Studio 버전은 v1.0.0 입니다. 설치와 제거는 8 장에서 다룹니다.

## 1.1 제품 구성과 시험 범위

NeoFlow Studio 의 기능은 Talend Open Studio 8.0.1 이 제공하는 기본 기능과, 당사(주식회사 위블리스)가 자체 개발하여 추가한 NeoFlow 전용 기능으로 나뉩니다. 시험 범위는 Talend Open Studio 8.0.1 이 제공하는 기본 기능을 제외하고, 당사가 자체 개발하여 추가한 기능으로 한정합니다. 다만 자체 개발 기능과 연계되는 기본 기능(예: Job 실행·빌드)은 연동 확인 수준에서 시험 범위에 포함할 수 있습니다.

| 구분 | 내용 | 시험 범위 |
|---|---|---|
| 자체 개발 기능 | 당사가 자체 개발하여 추가한 NeoFlow 전용 기능 — 1.2 자체 개발 기능 목록의 14 개 기능 | 포함 |
| 연동 기본 기능 | 자체 개발 기능과 연계되어 동작하는 기본 기능 — Job 실행, Job 빌드(Build Job), Repository 메뉴 | 연동 확인 수준에서 포함 |
| 기본 기능 | Talend Open Studio 8.0.1 이 제공하는 그 밖의 기능 — 표준 컴포넌트(tMap·tDBInput 등), Job 디자이너, Repository·메타데이터, 코드 생성 | 제외 |

### 연동 확인 대상 기본 기능

연동 확인은 기본 기능 자체를 검증하지 않고, 자체 개발 기능이 그 기본 기능 안에서 정상 동작하는지만 확인합니다.

| 기본 기능 | 연계되는 자체 개발 기능 | 연동 확인 내용 |
|---|---|---|
| Job 실행 | tNeoFlowPre · tNeoFlowPost · tNeoFlowThread, NeoFlowContext, NeoFlowJobApi | NeoFlow 컴포넌트·routine 을 넣은 Job 이 Studio 에서 코드 생성·빌드되는지 확인. BizDate 구간·파라미터 수신과 워터마크 커밋은 Sync·활성화한 Job 을 NeoFlow Server 웹 콘솔에서 실행해 확인(시험 환경은 운영 프로파일이라 Studio 로컬 실행에서는 서버 값을 받지 않음, 3.6) |
| Job 빌드 (Build Job) | Build Job NeoFlow Deploy, 라이선스 사용 제한 | Deploy 를 켜고 빌드하면 빌드 zip 이 선택한 서버에 업로드되고 결과 창이 뜨는지, 라이선스가 없으면 빌드가 차단되는지 확인 |
| Repository 메뉴 | Sync to NeoFlow Server, Job 사본 연결 해제 | Job 우클릭 메뉴에서 Sync 가 실행되는지, Duplicate·Copy/Paste 사본의 NeoFlow 연결 값이 비워지는지 확인 |

## 1.2 자체 개발 기능 목록

당사가 자체 개발하여 추가한 기능은 다음 14 개입니다. 「참조」 열은 이 매뉴얼의 설명 위치입니다.

| 번호 | 분류 | 기능 | 주요 내용 | 진입 경로 | 참조 |
|---|---|---|---|---|---|
| 1 | 라이선스 | License 등록 | 라이선스 키 검증(Validation)·등록(Register), 상태·만료일 표시, license.bin 암호화 저장 | [Help] → [License Registration] | 2 장 |
| 2 | 라이선스 | 라이선스 사용 제한 | 라이선스가 없거나 유효하지 않으면 Sync·Build Job 실행을 막고 Error 창 표시 | Sync·Build Job 실행 시 | 2.3 |
| 3 | 서버 연결 | NeoFlow Server 프로파일 관리 | 프로파일 추가·삭제·기본값 지정, Base URL·API token(암호화)·Build context·타임아웃 입력 | [Window] → [Preferences] → [NeoFlow] → [NeoFlow Server] | 3.2~3.4 |
| 4 | 서버 연결 | 연결 테스트 | 서버 도달 여부와 API token 권한 확인 | [Test Connection] 버튼 | 3.5 |
| 5 | 서버 연결 | NeoFlow 연결 정보 주입 | Sync·Deploy 때 서버 연결 변수(NEOFLOW_*)를 context 에 기록 | Sync·Deploy 실행 시 자동 | 3.6 |
| 6 | 동기화·배포 | Sync to NeoFlow Server | 서버·Job 선택 → Preview → Result 마법사로 서버 등록·빌드·업로드 | Job 우클릭 → [Sync to NeoFlow Server...] | 6 장 |
| 7 | 동기화·배포 | 열린 Job 편집기 정리 | Sync·Deploy 전에 대상 Job 의 편집기를 저장하고 닫음, 저장을 취소하면 중단 | Sync·Deploy 실행 시 자동 | 6.4 |
| 8 | 동기화·배포 | Job 사본 연결 해제 | Duplicate·Copy/Paste 사본의 NEOFLOW_* 값을 비워 원본의 서버 Job 과 분리 | [Duplicate], [Copy] → [Paste] | 6.4 |
| 9 | 동기화·배포 | Build Job NeoFlow Deploy | Build Job 화면의 NeoFlow Deploy 영역(서버 선택·Deploy·routines.jar 만 포함), 결과 창, 비모달 창 | Repository 우클릭 → [Build Job] | 7 장 |
| 10 | 컴포넌트 | tNeoFlowPre | 서버에서 BizDate 구간·적재 유형·실행 ID 를 받아 globalMap 에 채움 | 팔레트 NeoFlow/BizDate | 4.1 |
| 11 | 컴포넌트 | tNeoFlowPost | 처리를 마친 구간의 BizDate 워터마크를 서버에 커밋 | 팔레트 NeoFlow/BizDate | 4.2 |
| 12 | 컴포넌트 | tNeoFlowThread | 연결한 서브잡을 병렬 스레드로 실행, 기동 간격·Die on error 설정 | 팔레트 NeoFlow/Process | 4.3 |
| 13 | Job 파라미터 | NeoFlowContext | 서버가 넣어 준 PARAM_* 환경변수를 context 변수에 반영하는 시스템 routine | [Code] → [Routines] → [system] | 5.1~5.4 |
| 14 | Job 파라미터 | NeoFlowJobApi | Job 안에서 서버 파라미터·실행 컨텍스트 조회(getParameter 등) | 컴포넌트 입력 칸 ·tJava | 5.5 |

> **참고** 다음은 기능이 아니므로 목록에 넣지 않았습니다 — 제품 이름·시작 화면·About 창·환경설정 최상위 항목 이름(NeoFlow) 같은 브랜딩 표기, 기본 기능의 오류 수정. API token 발급(3.1)은 NeoFlow Server 관리 콘솔의 기능입니다.

## 1.3 전체 작업 흐름

NeoFlow Studio 로 ETL Job 을 서버에 올리기까지의 표준 흐름은 다음과 같습니다.

1. 설치 — 배포 압축 파일을 풀고 `NEO_BD-win-x86_64.ini` 에 JDK 11 경로를 지정한 뒤 실행합니다. (8 장)
2. License 등록 — 상단 Help 메뉴에서 발급받은 라이선스 키를 등록합니다. (2 장)
3. NeoFlow Server 프로파일 등록 — 관리 콘솔에서 API token 을 발급받고, Preferences 에서 서버 Base URL 과 함께 설정합니다. (3 장)
4. Job 작성 — 팔레트의 NeoFlow 컴포넌트(`tNeoFlowPre`/`tNeoFlowPost`/`tNeoFlowThread`)로 ETL Job 을 설계합니다. `tNeoFlowPre` 는 `tPrejob` 뒤, `tNeoFlowPost` 는 `tPostjob` 뒤에 배치합니다. (4 장)
5. 파라미터 받기 — 서버 파라미터를 context 변수로 받으려면 시스템 routine `NeoFlowContext` 를 `tPrejob` → `tJava` 에서 한 줄 호출하고, 컴포넌트 입력 칸에서 바로 쓰려면 `NeoFlowJobApi.getParameter()` 를 씁니다. (5 장)
6. Sync to NeoFlow Server — Repository 에서 Job 을 우클릭해 서버로 동기화·업로드합니다. (6 장)

> **주의** License 가 유효하지 않으면 Sync 및 Build Job 메뉴가 동작하지 않습니다. 가장 먼저 License 를 등록하세요.

## 1.4 이 매뉴얼 보는 법

본문에서 메뉴 경로는 [메뉴] → [하위 메뉴] 형태로, 컴포넌트·코드·context 변수는 `tNeoFlowPre`, `context.NEOFLOW_BASE_URL` 처럼 고정폭 글꼴로 표기합니다. 중요한 내용은 아래 세 가지 안내 상자로 강조합니다.

> **참고** 참고 — 알아두면 도움이 되는 부가 정보입니다.

> **TIP** TIP — 더 효율적으로 사용하는 방법입니다.

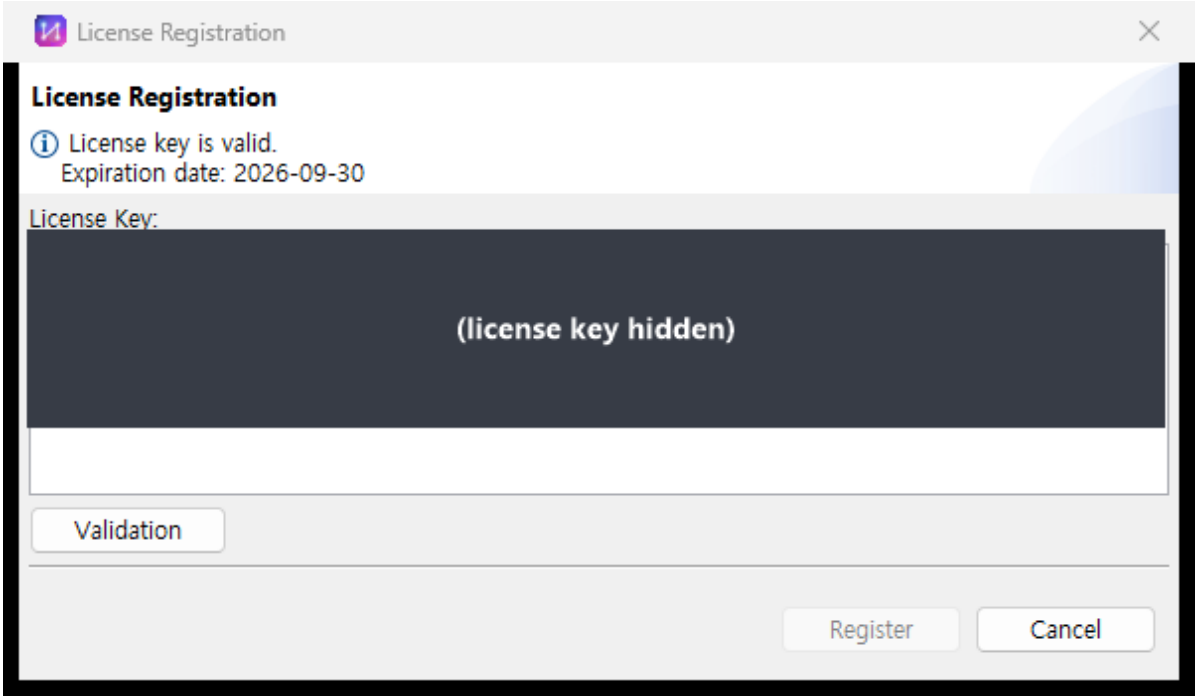
> **주의** 주의 — 데이터 손실·오류를 유발할 수 있어 각별히 주의해야 하는 항목입니다.

# 2. License 등록

NeoFlow Studio 의 핵심 기능(**Sync to NeoFlow Server**, **Build Job**)은 유효한 라이선스가 있어야 동작합니다. 라이선스가 없거나 만료되면 해당 메뉴 실행 시 에러 다이얼로그가 뜨고 진행되지 않습니다.

## 2.1 라이선스 등록 방법

1. 상단 메뉴 [Help] → [License Registration] 을 선택합니다.
2. License Registration 다이얼로그가 열립니다. 상단에 현재 라이선스 상태가 표시됩니다.
3. 발급받은 라이선스 키 전문을 License Key 입력란에 그대로 붙여넣습니다. 키 일부만 복사되면 검증에 실패하므로 반드시 전체를 복사하세요.
4. Validation 버튼을 눌러 키를 검증합니다. 정상이면 `License key is valid.` 와 만료일(Expiration date)이 표시되고 Register 버튼이 활성화됩니다.
5. Register 버튼을 눌러 라이선스를 저장합니다.




▲ *License Registration — 키 검증 성공 시 만료일 표시 + Register 활성화*

## 2.2 라이선스 저장 위치

등록된 라이선스는 Studio 설치 폴더에 `license.bin` 파일(경로: `<NeoFlow Studio 설치 폴더>/license.bin`)로 암호화되어 저장되며, 다음 정보를 포함합니다.

| 항목 | 설명 | 예시 |
|---|---|---|
| `productName` | 제품명 (Studio 제품명과 일치해야 함) | `NeoFlow Studio` |
| `majorVersion` | 제품 버전 (Studio 버전과 일치해야 함) | `1.0.0` |
| `macAddress` | 사용 PC 의 MAC 주소 (전체 사용 시 ALL) | `AA-BB-CC-DD-EE-FF / ALL` |
| `expirationDate` | 만료일 (yyyy-MM-dd) | `2027-12-31` |

> **참고** `macAddress` 가 특정 값으로 발급된 라이선스는 해당 PC 에서만 유효합니다. PC 를 교체하면 재발급이 필요합니다(`ALL` 로 발급된 경우 제외).

## 2.3 자주 보는 메시지

License Registration 다이얼로그 상단에는 현재 상태와 [Validation] 결과가 다음 문구로 표시됩니다. <만료일> 자리에는 라이선스의 만료일(yyyy-MM-dd)이 들어갑니다.

| 상황 | 메시지 (화면 표시 그대로) |
|---|---|
| 라이선스 미등록 | `Please register license key.` |
| 키 검증 실패(키 일부만 복사 등) | `License key validation failed. Please check that copied and pasted the whole key. If the problem persists, please contact We-Bliss Inc.` |
| 제품명 불일치 | `License key validation failed(Product). Please check that copied and pasted the whole key. If the problem persists, please contact We-Bliss Inc.` |
| 버전 불일치 | `License key validation failed(Version). Please check that copied and pasted the whole key. If the problem persists, please contact We-Bliss Inc.` |
| MAC 주소 확인 불가 | `Check the network for MAC address information.` |
| MAC 주소 불일치 | `The current MAC address and the MAC address registered in the license are different.` |
| 만료됨 | `License date(<만료일>) expired. Please register new license key` |
| 키를 고친 뒤 검증 전 | `License key modified. Please register after validation.` |
| 정상 | `License key is valid. Expiration date: <만료일>` |

라이선스가 없거나 유효하지 않은 상태에서 [Sync to NeoFlow Server...] 나 [Build Job] 을 실행하면 제목이 `Error` 인 창에 다음 문구가 뜨고 메뉴가 진행되지 않습니다.

| 상황 | 메시지 (화면 표시 그대로) |
|---|---|
| 라이선스 미등록 | `Please register license key.` |
| 키 손상·검증 실패 | `License key validation failed. Please check that copied and pasted the whole key. If the problem persists, please contact We-Bliss Inc.` |
| 만료됨 | `License date expired. Please register new license key. If the problem persists, please contact We-Bliss Inc.` |

> **참고** 메시지 속 `We-Bliss Inc.` 는 제조사 주식회사 위블리스의 영문 표기입니다. 문제가 지속되면 라이선스 발급처(주식회사 위블리스)에 문의하세요.

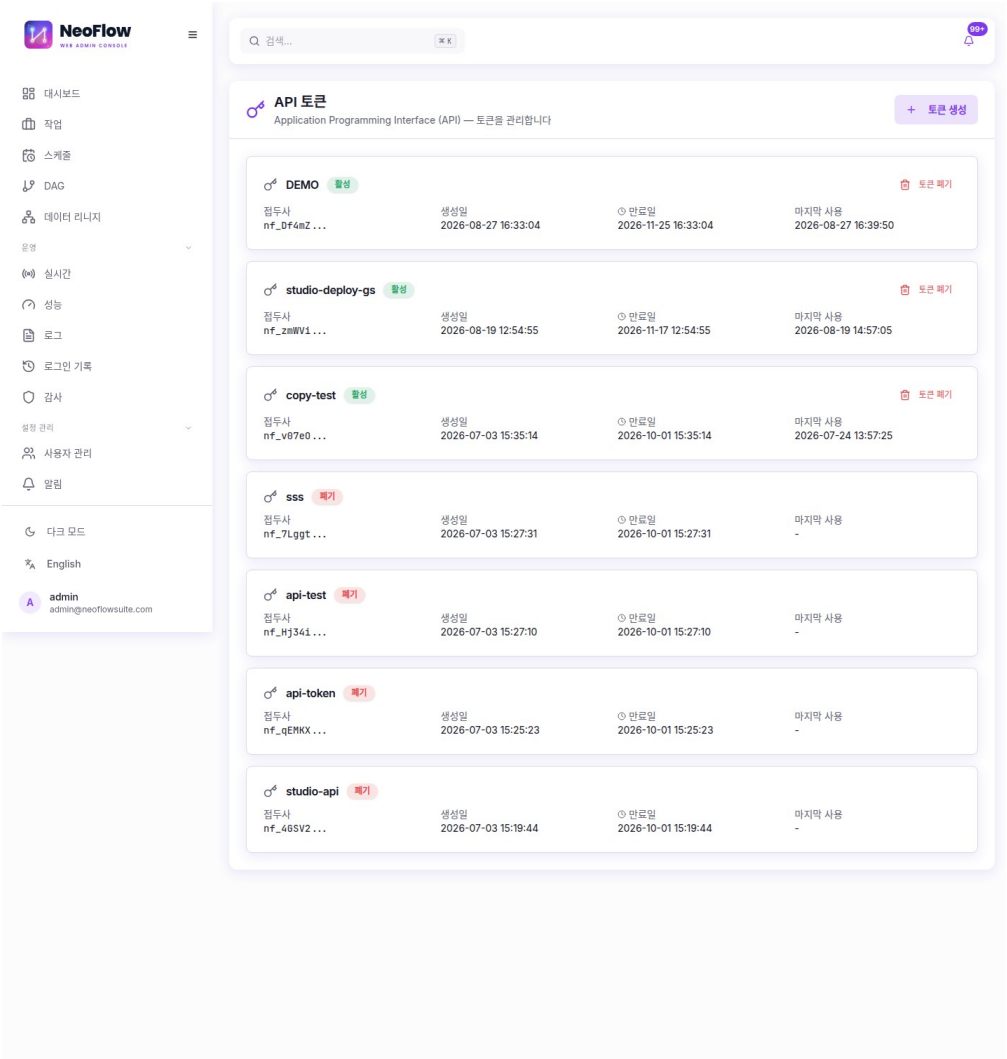
# 3. NeoFlow Server 연결 정보 등록

Sync 기능이 Job 을 빌드해서 보낼 NeoFlow 서버 프로파일을 먼저 등록합니다. 프로파일은 여러 개(예: `LOCAL`, `DEV`, `PROD`) 만들 수 있고, 그중 하나를 기본값으로 지정합니다. 프로파일에 넣을 API token 은 NeoFlow Server 관리 콘솔에서 먼저 발급받아야 하며(3.1), Studio 에 저장될 때 암호화됩니다.

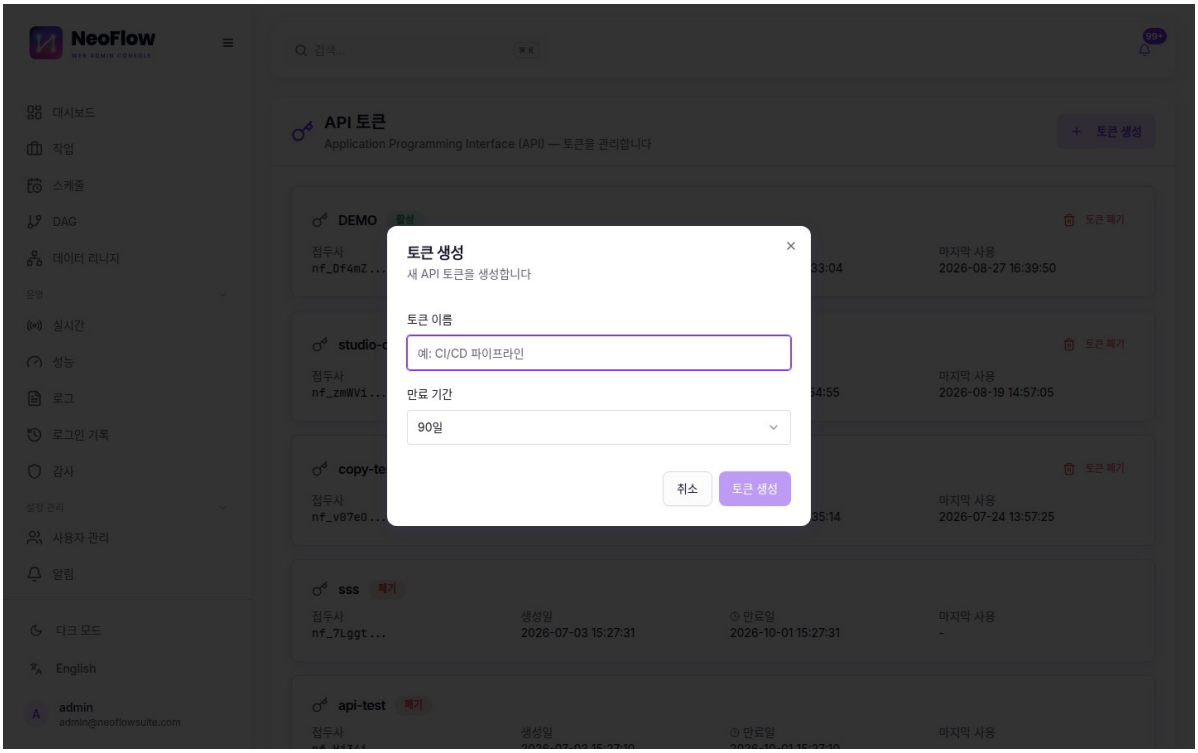
## 3.1 API token 발급 (NeoFlow Server)

프로파일에 입력할 API token 은 NeoFlow Server 웹 관리 콘솔에서 발급합니다. [API 토큰] 화면은 모든 사용자가 쓸 수 있고 각자 본인 토큰만 봅니다. 토큰은 발급한 사용자의 현재 역할로 동작하며, Sync 가 호출하는 동기화 API 는 ADMIN·매니저(JOB_MANAGER)만 허용하므로 Sync 용 토큰은 이 두 역할의 계정으로 발급해야 합니다.

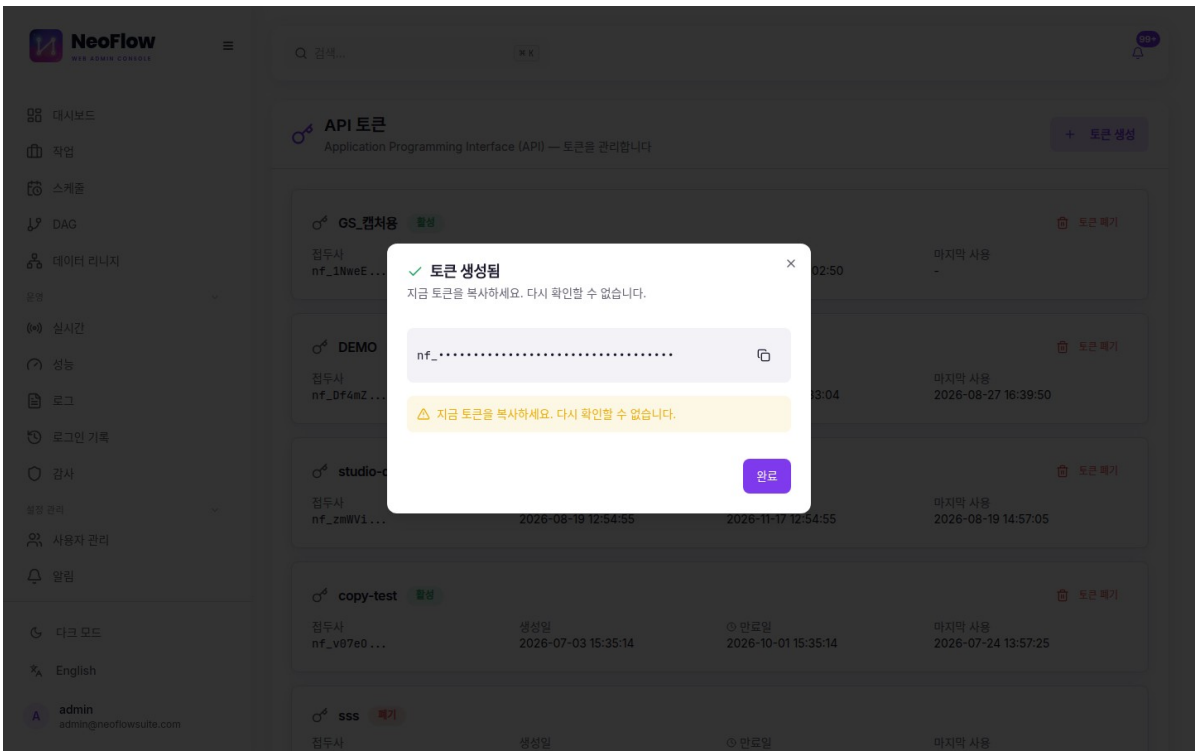
1. NeoFlow Server 웹 관리 콘솔에 ADMIN 또는 매니저 계정으로 로그인합니다. 초기 비밀번호로 처음 로그인하면 비밀번호 변경 화면이 먼저 나타나며, 비밀번호를 바꾸기 전에는 다른 화면을 쓸 수 없습니다.
2. 좌측 사이드바 설정 관리 영역의 [설정] → [API 토큰] 으로 이동합니다.
3. 화면 우측 상단의 [토큰 생성] 버튼을 누릅니다.
4. 토큰 이름(예: `NeoFlow_Studio`)과 만료 기간(30 일 / 60 일 / 90 일 / 만료 없음, 기본 90 일)을 지정하고 [토큰 생성] 을 누릅니다.
5. 토큰 생성됨 모달에 평문 토큰이 표시됩니다. 복사 버튼으로 즉시 복사한 뒤 [완료] 로 닫습니다.




*◢ NeoFlow Server 관리 콘솔 — 설정 › API 토큰(토큰 카드 목록 · 토큰 생성)*




▲ *토큰 생성 — 토큰 이름 · 만료 기간(30 일 / 60 일 / 90 일 / 만료 없음) 지정*




▲ *토큰 생성됨 — 평문 토큰은 이때 한 번만 표시됩니다(문서에서는 토큰 값 마스킹)*

> **주의** 평문 토큰은 발급 직후 한 번만 표시되며 모달을 닫으면 다시 볼 수 없습니다. 복사하지 못했다면 해당 토큰을 [토큰 폐기] 하고 새로 발급하세요.

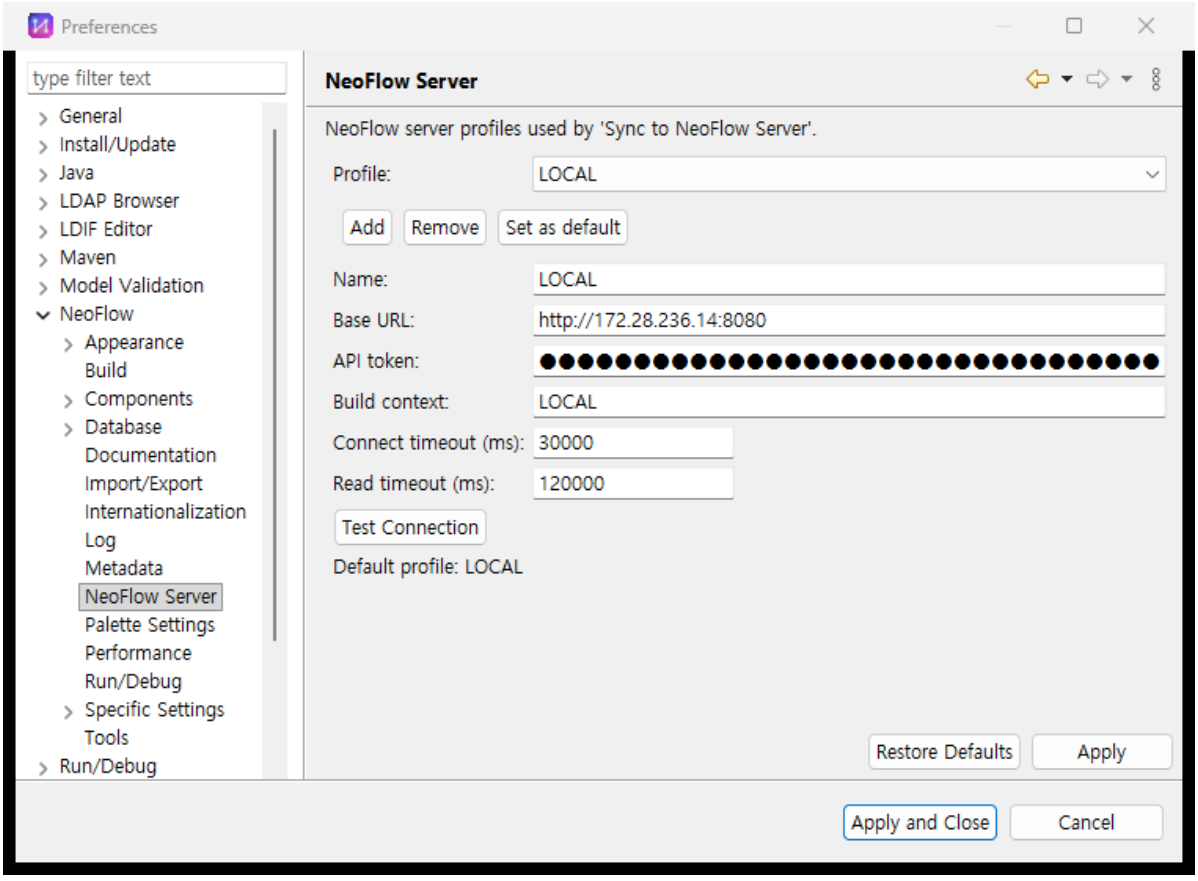
목록 화면의 토큰 카드에는 다음 정보가 표시됩니다.

| 항목 | 설명 |
|---|---|
| 이름 | 토큰 식별용 이름 |
| 상태 | 활성 / 만료 / 폐기 |
| 접두사 | 토큰 앞부분(nf_ + 일부). 전체 토큰 값은 표시되지 않습니다. |
| 생성일 / 만료일 | 발급 일시 및 만료 일시 |
| 마지막 사용 | 해당 토큰으로 마지막 인증한 일시(최대 5 분 늦게 반영, 미사용 시 - 로 표시) |

> **참고** 발급한 토큰 값은 3.4 프로파일별 입력 항목의 `API token` 란에 붙여넣습니다.

## 3.2 Preferences 페이지 열기

상단 메뉴 [Window] → [Preferences] 를 열고, 왼쪽 트리에서 [NeoFlow] → [NeoFlow Server] 를 선택합니다.




▲ *Preferences → NeoFlow → NeoFlow Server — 프로파일 관리 · 입력 항목 · 전역 타임아웃 · Test Connection*

## 3.3 프로파일 관리

페이지 상단에서 프로파일을 추가·삭제하고 기본값을 지정합니다. 하단에는 `Default profile: <이름>` 으로 현재 기본 프로파일이 표시됩니다.

| 컨트롤 | 설명 |
|---|---|
| Profile (드롭다운) | 등록된 프로파일 목록에서 선택 |

| 컨트롤 | 설명 |
|---|---|
| Add | 새 프로파일 추가 (PROFILE_1, PROFILE_2... 자동 명명) |
| Remove | 선택한 프로파일 삭제 |
| Set as default | 선택한 프로파일을 기본값으로 지정 (Sync 마법사가 기본 선택) |

## 3.4 프로파일별 입력 항목

각 프로파일에는 서버 주소와 인증 정보, 빌드 환경을 입력합니다.

| 필드 | 설명 | 예시 |
|---|---|---|
| Name | 프로파일 이름 | `PROD` |
| Base URL | NeoFlow 서버 주소 (스킴+호스트, 기본 포트가 아니면 포트 포함) | `https://neoflow.example.com` |
| API token | 서버 인증용 Bearer 토큰 (마스킹·암호화 저장) | (서버에서 발급) |
| Build context | 빌드 시 사용할 context 이름. 비우면 프로파일 이름 사용 (아래 참고) | `Default` |

타임아웃은 모든 프로파일에 공통 적용되는 전역 설정입니다.

| 필드 | 기본값 | 설명 |
|---|---|---|
| Connect timeout (ms) | `5000` | 서버 연결 타임아웃 |
| Read timeout (ms) | `30000` | 응답 읽기 타임아웃 |

값을 입력·수정한 뒤에는 [Apply] 또는 [Apply and Close] 를 눌러야 저장됩니다.

> **참고** Build context 는 Sync 할 Job 에 이미 있는 context 이름으로 지정하세요. Job 에 없는 이름이면 Sync 가 그 Job 의 첫 번째 context 를 복사해 같은 이름의 환경을 새로 만들고 기본 context 로 지정합니다. 그 결과 PC 경로처럼 서버에서 쓸 수 없는 값이 서버 실행에 그대로 들어갈 수 있습니다.

> **참고** GS 인증 시험 환경에서는 Base URL 에 로드밸런서 주소(https://<로드밸런서 공인 IP>)를, Build context 에 `DEMO` 를 입력합니다. 함께 전달한 시험용 Job 의 context 가 DEMO 하나뿐이기 때문입니다.

## 3.5 연결 테스트

Base URL 과 API token 을 모두 입력한 뒤 Test Connection 버튼을 누르면 서버 도달 여부와 토큰 권한을 확인합니다.

| 상황 | 메시지 (제목 Test Connection 창, 화면 표시 그대로) |
|---|---|
| 성공 | `Success: server reachable and token authorized.` |
| Base URL·API token 미입력 | `Enter base URL and API token first.` |
| 토큰 만료·폐기·오입력, 발급 계정 비활성 | `Invalid or expired API token` |

| 상황 | 메시지 (제목 Test Connection 창, 화면 표시 그대로) |
|---|---|
| 토큰 발급 계정이 ADMIN·매니저가 아님 | `Token lacks the ADMIN/JOB_MANAGER role` |
| 서버에 연결할 수 없음(주소·방화벽·인증서) | `Cannot reach server: <원인>` |

> **주의** 서버가 자체 서명 인증서(self-signed)를 쓰면 `Cannot reach server:` 뒤에 `PKIX path building failed` 가 표시됩니다. Studio 는 실행에 쓰는 JDK 의 인증서 저장소(`cacerts`)로만 서버 인증서를 확인하므로, 서버 관리자에게 받은 인증서 파일(예: `tls.crt`)을 이 저장소에 등록한 뒤 Studio 를 다시 시작해야 합니다. Sync·Deploy 도 같은 이유로 실패합니다.

관리자 권한 명령 프롬프트에서 아래처럼 등록합니다. `JAVA_HOME` 은 Studio ini 의 `-vm` 에 지정한 JDK 폴더로, 인증서 경로는 실제 위치로 바꿉니다. 첫 명령이 보여 주는 SHA-256 지문을 서버 관리자에게 받은 값과 대조한 뒤 등록하세요.

```
set JAVA_HOME=C:\Program Files\Java\jdk-11
"%JAVA_HOME%\bin\keytool" -printcert -file C:\neoflow\tls.crt
"%JAVA_HOME%\bin\keytool" -importcert -alias neoflow-server ^
    -file C:\neoflow\tls.crt ^
    -keystore "%JAVA_HOME%\lib\security\cacerts" -storepass changeit -noprompt
```

> **참고** GS 인증 시험 환경의 로드밸런서는 자체 서명 인증서를 씁니다. 함께 전달한 `tls.crt` 를 위 방법으로 등록하세요.

## 3.6 저장 및 Job 으로의 주입 원리

프로파일은 NeoFlow Studio 환경설정(Preferences)에 저장되며 API token 은 암호화되어 보관됩니다. Sync 실행 시 선택한 프로파일의 정보가 Job 의 context 변수로 주입됩니다(프로파일의 `Build context` 환경에 기록).

| context 변수 | 값 | Job 에서 참조 |
|---|---|---|
| `NEOFLOW_BASE_URL` | 서버 Base URL | `context.NEOFLOW_BASE_URL` |
| `NEOFLOW_API_TOKEN` | API 토큰 | `context.NEOFLOW_API_TOKEN` |
| `NEOFLOW_JOB_ID` | Sync 로 발급된 NeoFlow Job ID | `context.NEOFLOW_JOB_ID` |

> **참고** 이 변수들은 `tNeoFlowPre` 의 Advanced settings(`BASE_URL`·`API_TOKEN`·`JOB_ID`)에 연결되며, Studio 에서 Job 을 직접 실행(테스트 실행)할 때만 쓰입니다. 사용자가 직접 입력할 필요는 없으며, Sync 가 자동으로 채워 넣습니다. NeoFlow Server 에서 실행될 때는 서버가 환경변수로 넣어 준 값이 우선하므로 이 context 값은 쓰이지 않습니다.

**참고** `tNeoFlowPost` 에는 `BASE_URL`·`API_TOKEN`·`JOB_ID` 설정 항목이 없습니다. NeoFlow Server 에서 실행될 때는 서버가 실행마다 발급해 환경변수로 넣어 주는 실행 토큰(`NEOFLOW_TOKEN`)과 실행 ID(`EXECUTION_ID`)로 워터마크를 커밋하고, Studio 테스트 실행에서는 같은 실행에서 `tNeoFlowPre` 가 연 세션을 이어 씁니다.

**주의** Studio 의 테스트 실행(로컬 실행)에서 서버 값을 받는 것은 개발·검증 서버에서만 됩니다. 로컬 실행 때 `tNeoFlowPre` 가 서버에 개발용 실행 토큰(`/api/executions/dev-token`)을 요청하는데, 운영(prod) 프로파일로 설치한 서버는 이 요청을 403 으로 거부합니다. 이때 Job 자체는 실행되지만 `tNeoFlowPre` 는 BizDate 값을 채우지 않고 통과하고, `tNeoFlowPost` 는 커밋을 건너뛰며, `NeoFlowJobApi` 를 호출하는 Job 은 실패합니다. `NeoFlowContext` 만 쓰는 Job 은 Context 탭 기본값으로 실행됩니다. 운영 서버에서는 Job 을 Sync·활성화한 뒤 NeoFlow Server 웹 콘솔에서 실행하세요.

**참고** GS 인증 시험 환경의 NeoFlow Server 는 운영(prod) 프로파일로 설치되어 있습니다. 따라서 BizDate 구간·서버 파라미터 수신과 워터마크 커밋은 Studio 로컬 실행이 아니라, Sync·활성화한 Job 을 웹 콘솔에서 실행해 확인합니다.

# 4. NeoFlow 컴포넌트

NeoFlow 전용 컴포넌트는 팔레트의 NeoFlow 폴더에 있습니다. BizDate 처리용 두 컴포넌트와 병렬 실행용 한 컴포넌트로 구성됩니다.




▲ 팔레트 NeoFlow 패밀리 — BizDate(tNeoFlowPost·tNeoFlowPre) · Process(tNeoFlowThread)

| 컴포넌트 | 팔레트 위치 | 용도 |
|---|---|---|
| tNeoFlowPre | NeoFlow/BizDate | 서버에서 처리 기준일(BizDate) 구간을 가져옴 |
| tNeoFlowPost | NeoFlow/BizDate | 처리 완료된 BizDate 워터마크를 서버에 커밋 |
| tNeoFlowThread | NeoFlow/Process | 연결된 서브잡들을 병렬 스레드로 실행(시차 기동) |

> **참고** 세 컴포넌트 모두 데이터 플로우(행) 연결이 없는 트리거 전용 시작(STARTABLE) 컴포넌트입니다. OnSubjobOk, OnComponentOk, RunIf 등 트리거 링크로만 연결합니다.

### 배치 규칙 — tPrejob / tPostjob 과의 순서

BizDate 컴포넌트 두 개는 Job 의 사전·사후 구간에 각각 배치합니다. 순서가 어긋나면 BizDate 구간이 채워지기 전에 본 처리가 실행되거나, 처리가 끝나기 전에 워터마크가 커밋될 수 있습니다.

- tNeoFlowPre 는 tPrejob 컴포넌트 뒤에 배치합니다 — Job 의 본 처리보다 먼저 실행되어 BizDate 구간을 globalMap 에 채워야 합니다.
- tNeoFlowPost 는 tPostjob 컴포넌트 뒤에 배치합니다 — 본 처리가 모두 끝난 뒤 워터마크를 커밋해야 합니다.
- 두 연결 모두 OnComponentOk 트리거 링크를 사용합니다.

```
tPrejob   --OnComponentOk-->  tNeoFlowPre   --OnComponentOk-->  [ main subjob ]
tPostjob  --OnComponentOk-->  tNeoFlowPost  --OnComponentOk-->  [ close / cleanup ]
```

> **참고** tPrejob / tPostjob 은 Talend Open Studio 표준 컴포넌트로, 각각 Job 시작 전과 종료 후에 실행되는 구간을 정의합니다. NeoFlow BizDate 컴포넌트는 이 구간의 첫 번째 컴포넌트로 두는 것을 권장합니다.

## 4.1 tNeoFlowPre — BizDate 구간 가져오기

### 역할

NeoFlow 서버에서 이번 실행이 처리해야 할 기준일 구간(시작/종료), 로드 타입, 실행 ID 를 가져와 globalMap 에 채워 넣습니다. Job 파라미터는 context 변수에 넣지 않습니다(파라미터는 5 장). tPrejob 컴포넌트 뒤에 배치하여 Job 의 본 처리보다 먼저 실행되게 합니다. 이후 SQL 컴포넌트 등에서 시작/종료 일자를 WHERE 조건으로 사용합니다.

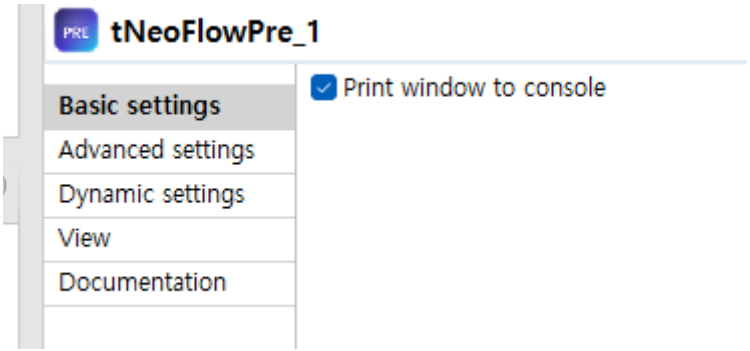
> **참고** Never-fail 원칙 — 서버에 연결되지 않거나 바인딩이 없어도 Job 을 실패시키지 않습니다. 경고만 로그로 남기고 진행합니다(이 경우 BizDate 값이 채워지지 않음).

### 동작 우선순위

1. 서버 오케스트레이션 실행 — 환경변수/시스템 프로퍼티 `NEOFLOW_BASE_URL`·`NEOFLOW_TOKEN`·`EXECUTION_ID` 가 모두 있으면 그 실행 컨텍스트를 사용합니다(아래 Advanced 파라미터 무시).
2. Studio 테스트 실행 — 위 환경이 없으면 컴포넌트에 설정된 Base URL/Token/Job ID(보통 `context.NEOFLOW_*`)로 서버에서 값을 가져옵니다. 이 경로는 개발·검증 서버에서만 동작합니다. 운영(prod) 서버는 개발 토큰을 거부하므로 값을 받지 못하고 다음 단계로 넘어갑니다(3.6).
3. 둘 다 없으면 경고만 남기고 통과합니다.

### 파라미터 — Basic settings

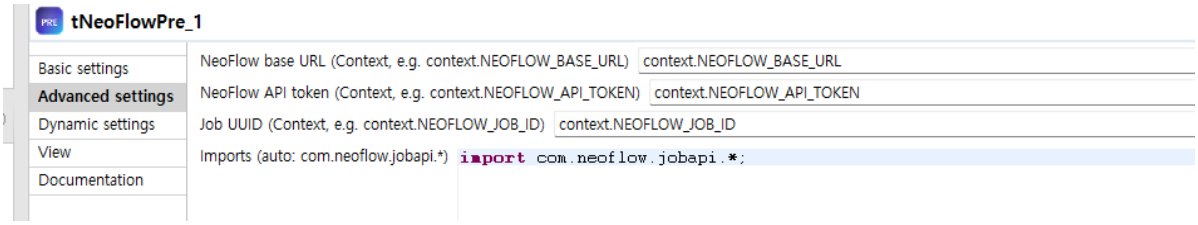
| 파라미터 | 라벨 | 기본값 | 설명 |
|---|---|---|---|
| `PRINT_LOG` | Print window to console | `true` | 체크 시 가져온 구간을 콘솔에 출력 |




▲ *tNeoFlowPre Basic settings — Print window to console 체크박스(기본 체크)*

### 파라미터 — Advanced settings

| 파라미터 | 라벨 | 기본값 | 설명 |
|---|---|---|---|
| `BASE_URL` | `NeoFlow base URL (Context, e.g. context.NEOFLOW_BASE_URL)` | (빈 값) | Studio 테스트용 서버 URL. Sync/Deploy 시 자동 입력, 서버 실행 시 비워둠 |
| `API_TOKEN` | `NeoFlow API token (Context, e.g. context.NEOFLOW_API_TOKEN)` | (빈 값) | Studio 테스트용 API 토큰. 자동 입력 |
| `JOB_ID` | `Job UUID (Context, e.g. context.NEOFLOW_JOB_ID)` | (빈 값) | Studio 테스트용 Job UUID. 자동 입력 |
| `IMPORT` | `Imports (auto: com.neoflow.jobapi.*)` | (자동) | import com.neoflow.jobapi.* — 자동 생성, 수정 불필요 |




▲ *tNeoFlowPre Advanced settings — 세 값은 context.NEOFLOW_* 로 바인딩(Sync 가 자동 채움)*

> **참고** Advanced 의 세 값은 일반적으로 직접 입력하지 않습니다. Sync 가 `context.NEOFLOW_*` 로 자동 채웁니다.

### 반환값 (globalMap)

컴포넌트 실행 후(`AFTER`) 사용할 수 있는 반환값입니다. `cid` 는 Job 내 컴포넌트 고유 이름입니다.

| 반환명 | 타입 | globalMap 키 |
|---|---|---|
| `START_DATE` | Object(LocalDateTime) | `<cid>_START_DATE` |
| `END_DATE` | Object(LocalDateTime) | `<cid>_END_DATE` |
| `LOAD_TYPE` | String | `<cid>_LOAD_TYPE` |
| `EXECUTION_ID` | Object | `<cid>_EXECUTION_ID` |

다운스트림에서 바로 쓰는 고정 globalMap 키:

| 키 | 타입 | 설명 |
|---|---|---|
| `nf_endDate` | LocalDateTime | 구간 종료일 (tNeoFlowPost 가 읽음) |
| `nf_startStr` | String (yyyy-MM-dd HH:mm:ss) | 포맷된 시작일 |
| `nf_endStr` | String (yyyy-MM-dd HH:mm:ss) | 포맷된 종료일 |

### 사용 예 — SQL 컴포넌트의 쿼리에서 구간 사용

```
SELECT * FROM sales
WHERE updated_at >= '" + (String)globalMap.get("nf_startStr") + "'
  AND updated_at <  '" + (String)globalMap.get("nf_endStr") + "'
```

## 4.2 tNeoFlowPost — BizDate 워터마크 커밋

### 역할

`tNeoFlowPre` 가 가져온 구간의 종료일(`nf_endDate`)을 서버에 커밋하여, 해당 로드 구간이 정상 처리되었음을 기록합니다. `tPostjob` 컴포넌트 뒤에 배치하여 데이터 적재가 모두 끝난 뒤 실행되게 합니다.

> **참고** Never-fail 원칙 — 커밋 실패, `nf_endDate` 누락, 예외 발생 시에도 Job 을 실패시키지 않고 로그만 남기고 통과합니다.

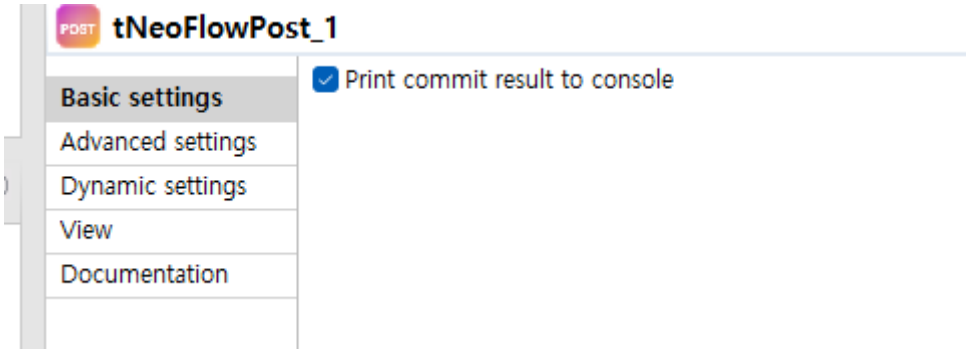
> **주의** 개발·검증 서버에 연결해 Studio 에서 실행하면 `tNeoFlowPost` 가 그 서버 Job 의 실제 워터마크를 커밋합니다. 다음 서버 실행은 이 시각부터 구간을 받으므로, 테스트 뒤 필요하면 NeoFlow Server 의 Job 상세 화면에서 워터마크를 되돌리세요.

### 전제 조건

- 같은 Job 에 `tNeoFlowPre` 가 먼저 실행되어 있어야 합니다. `tNeoFlowPost` 는 `tNeoFlowPre` 가 채운 고정 키 `nf_endDate` 를 읽습니다.
- `tNeoFlowPre` 가 실행되지 않았거나 바인딩이 없으면 `nf_endDate` 가 없어 커밋을 건너뜁니다(로그: "did tNeoFlowPre run?").
- 배치 순서 — `tNeoFlowPre` 는 `tPrejob` 뒤, `tNeoFlowPost` 는 `tPostjob` 뒤에 두어야 두 컴포넌트의 실행 순서가 보장됩니다.

### 파라미터 — Basic settings

| 파라미터 | 라벨 | 기본값 | 설명 |
|---|---|---|---|
| `PRINT_LOG` | Print commit result to console | `true` | 체크 시 커밋 결과(성공/스킵 사유)를 콘솔에 출력 |




▲ *tNeoFlowPost Basic settings — Print commit result to console(기본 체크). Advanced settings 항목 없음*

### 반환값 (globalMap)

| 반환명 | 타입 | globalMap 키 | 설명 |
|---|---|---|---|
| `COMMITTED` | Boolean | `<cid>_COMMITTED` | 커밋 성공 시 true, 스킵/실패 시 false |
| `SKIP_REASON` | String | `<cid>_SKIP_REASON` | 스킵 사유. NON_MONOTONIC = 종료 시각이 기존 워터마크보다 늦지 않음, BACKFILL = 소급 적재 실행. 정상 커밋·오류 시 빈 값 |

### 표준 배치 패턴

4 장 배치 규칙과 같습니다. `tNeoFlowPost` 는 본 처리 서브잡에 직접 잇지 않고 `tPostjob` 에 `OnComponentOk` 로 연결합니다.

```
tPrejob   --OnComponentOk-->  tNeoFlowPre   --OnComponentOk-->  [ main subjob ]
tPostjob  --OnComponentOk-->  tNeoFlowPost  --OnComponentOk-->  [ close / cleanup ]
```

## 4.3 tNeoFlowThread — 서브잡 병렬 실행(시차 기동)

### 역할

이 컴포넌트에 연결된 여러 서브잡을 각각 별도 스레드로 병렬 실행합니다. 스레드를 동시에 띄우지 않고 설정한 초만큼 시차(ramp-up)를 두고 순차적으로 기동할 수 있습니다(DB 커넥션 폭주 방지 등). 모든 스레드가 끝날 때까지 Job 은 대기합니다.

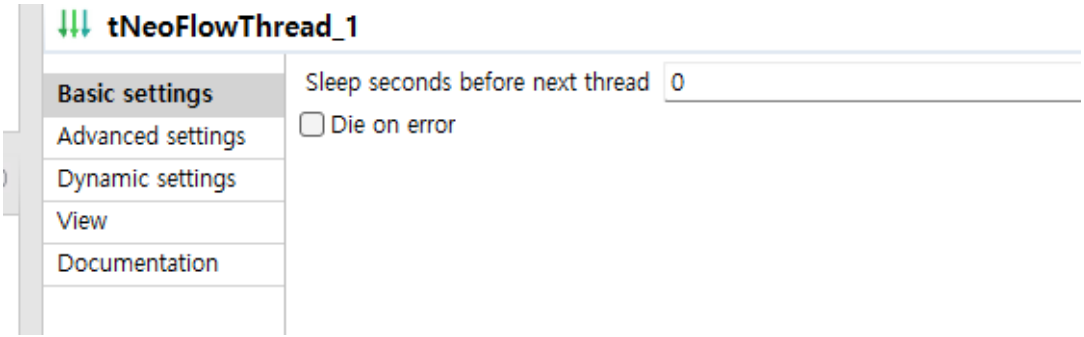
> **참고** `tNeoFlowPre/tNeoFlowPost` 와 달리 에러 처리는 옵션(`DIE_ON_ERROR`)으로 제어합니다. never-fail 이 아닙니다.

### 연결 방법 — THREAD_PROCESS 링크

- `tNeoFlowThread` 에서 병렬로 돌릴 각 서브잡 시작 컴포넌트로 녹색 점선 트리거(`THREAD_PROCESS`)를 연결합니다.
- 최대 100 개까지 연결 가능하며, 연결 1 개당 스레드 1 개가 생성됩니다.
- 연결한 순서대로 스레드가 기동되고, 각 기동 사이에 시차가 적용됩니다(첫 스레드 제외).

### 파라미터 — Basic settings

| 파라미터 | 라벨 | 기본값 | 설명 |
|---|---|---|---|
| `SLEEP_SECOND` | Sleep seconds before next thread | `0` | 다음 스레드 기동 전 대기(초). 0 이면 즉시 모두 기동. context 변수도 가능 |
| `DIE_ON_ERROR` | Die on error | `false` | 체크 시 스레드 중 하나라도 예외가 나면 Job 실패. 해제 시 로그만 남기고 계속 |




▲ *tNeoFlowThread Basic settings — Sleep seconds(기본 0) · Die on error(기본 해제)*

### 동작 순서

1. `THREAD_PROCESS` 로 연결된 각 서브잡을 별도 스레드로 생성(스레드명 `<cid>_thread_<index>`).
2. 두 번째 스레드부터 기동 전 `SLEEP_SECOND` 초 대기(시차).
3. 모든 스레드가 끝날 때까지 메인 Job 이 대기(폴링).
4. `DIE_ON_ERROR=true` 면 첫 예외 발생 시 Job 실패, false 면 스택트레이스만 출력하고 완료.
5. 완료 시 콘솔: `[<cid>] Multi-thread process complete (N threads).`

### 주의

- 모든 스레드가 같은 `globalMap` 을 공유합니다. 서브잡이 공유 상태를 쓴다면 thread-safe(예: `ConcurrentHashMap`) 처리가 필요합니다.
- 반환값(RETURNS) 없음, 별도 jar 의존성 없음.

### 사용 예

```
tNeoFlowThread
    ├─ THREAD_PROCESS → tDBInput_A → tFileOutput_A
    ├─ THREAD_PROCESS → tDBInput_B → tFileOutput_B
    └─ THREAD_PROCESS → tDBInput_C → tFileOutput_C
```

SLEEP_SECOND=2 로 두면 A → (2 초) → B → (2 초) → C 순으로 기동되어 동시 부하를 분산합니다.

# 5. Job 파라미터 받기 (NeoFlowContext · NeoFlowJobApi)

NeoFlow Server 의 Job 파라미터와 공통 설정(Global Properties) 값을 Job 안에서 받아 쓰는 방법을 다룹니다. context 변수로 받으려면 `NeoFlowContext` routine(5.2~5.4)을, 컴포넌트 입력 칸에서 바로 조회하려면 `NeoFlowJobApi`(5.5)를 씁니다. 같은 Job 을 기준일·대상 시스템만 바꿔 재사용할 때 필요합니다.

## 5.1 파라미터가 Job 에 도달하는 경로

NeoFlow 서버는 실행 시 확정된 파라미터를 두 경로로 Job 프로세스에 넘깁니다.

| 경로 | 형식 | 전달 대상 |
|---|---|---|
| 실행 인자 | `--context_param <키>=<값>` | 비-시크릿 파라미터 (공통 설정의 비밀 속성은 제외) |
| 환경변수 | `PARAM_<대문자 키>=<값>` | 전체 파라미터 (시크릿 포함) |

시크릿을 실행 인자로 넘기지 않는 이유는 커맨드라인이 같은 호스트의 다른 프로세스에서 `/proc/<pid>/cmdline` 으로 읽히기 때문입니다. 즉 **시크릿 파라미터는 환경변수 경로로만 도달**합니다.

시크릿은 키 이름에 `PASSWORD`·`SECRET`·`TOKEN`·`KEY` 가 들어 있는지(대소문자 무시, 단어 일부 포함)로 정해지며, Job 파라미터·공통 설정·실행할 때 입력한 값 모두에 적용됩니다(예: `SORT_KEY` 도 시크릿입니다). 여기에 더해 공통 설정(Global Properties)에서 비밀로 지정한 키도 이름과 관계없이 시크릿으로 다룹니다. 그래서 `DB_CONN_STR` 처럼 이 단어가 없는 비밀 속성도 환경변수로만 도착하며, 실행 인자에 기대는 Job 은 아래 routine 을 두지 않으면 값을 받지 못합니다.

실행 인자 경로는 Talend Open Studio 규칙에 따라 **파라미터 키와 context 변수명이 정확히 일치**해야 바인딩됩니다. 대소문자만 달라도 매칭되지 않습니다. 아래 `NeoFlowContext` routine 이 이 두 가지 빈틈(시크릿·대소문자)을 함께 메웁니다.

## 5.2 NeoFlowContext routine (기본 포함)

`NeoFlowContext` 는 NeoFlow 가 주입한 `PARAM_*` 환경변수를 같은 이름의 context 변수에 자동으로 매핑하는 routine 입니다. NeoFlow Studio 에 시스템 routine 으로 포함되어 있어 별도로 등록할 필요가 없습니다. Repository 뷰의 [Code] → [Routines] → [system] 폴더에서 확인할 수 있으며, 모든 Job 에서 import 없이 `NeoFlowContext.apply(context)` 로 호출합니다.

> **주의** 같은 이름(`NeoFlowContext`)의 사용자 routine 을 만들지 마세요. 시스템 routine 과 클래스 이름이 겹쳐 Job 빌드가 꼬일 수 있습니다. 이전에 매뉴얼 절차대로 사용자 routine 을 만들어 두었다면 삭제하고 시스템 routine 을 쓰세요.

## 5.3 Job 에서 사용

`tPrejob` 뒤에 연결한 `tJava` 에 다음 한 줄을 둡니다. 이 한 줄이 모든 파라미터를 처리하므로, 파라미터가 늘어나도 Job 을 고칠 필요가 없습니다.

```
NeoFlowContext.apply(context);
```

이후 Job 어디서든 `context.GS_DATA_DIR` 처럼 평소대로 쓰면 됩니다. Studio 에서 로컬 실행할 때는 `PARAM_*` 환경변수가 없으므로 Context 탭에 넣어 둔 기본값이 그대로 유지됩니다. 따라서 로컬용·서버용 분기 코드를 둘 필요가 없습니다.

routine 을 쓰지 않고 직접 읽으려면 파라미터마다 아래 보일러플레이트를 Job 에 넣어야 합니다.

```
String gsDataDir = System.getenv("PARAM_GS_DATA_DIR");
if (gsDataDir != null && !gsDataDir.isEmpty()) {
    context.GS_DATA_DIR = gsDataDir;
}
```

> **참고** 이 직접 읽기 방식도 정상 동작하며, `--context_param` 을 지원하지 않는 구버전 NeoFlow 서버에서도 동일하게 쓸 수 있습니다. 다만 파라미터 수만큼 코드가 늘어나므로 routine 사용을 권장합니다.

## 5.4 동작 규칙과 제약

routine 은 다음 규칙으로 동작합니다.

| 항목 | 동작 |
|---|---|
| 매칭 | PARAM_ 접두어를 뗀 이름과 context 변수명을 대소문자 무시로 비교(NeoFlow 가 키를 대문자화하므로) |
| 덮어쓰기 범위 | NeoFlow 가 실제로 주입한 변수만. 나머지는 기존 값 유지 |
| 지원 타입 | String · Integer · Long · Boolean · Double · Float · BigDecimal · Character (대응 primitive 포함) |
| 미지원 타입 | 건드리지 않음 — 실행 인자가 세팅한 값이나 Context 탭 기본값이 그대로 보존됨 |
| 파싱 실패 | 해당 변수만 건너뛰고 경고를 남긴 뒤 Job 은 계속 진행 |
| Properties 동기화 | context.getProperty("X") 로 읽는 컴포넌트를 위해 내부 map 도 함께 갱신 |
| 로깅 | 적용된 변수명만 출력하고 값은 출력하지 않음 |

> **주의** 파라미터 값을 `System.out.println` 으로 출력하지 마세요. Job 의 표준출력은 NeoFlow 실행 로그로 그대로 적재되며 이 경로에는 서버측 시크릿 마스킹이 적용되지 않으므로, 값을 찍으면 패스워드 파라미터가 평문으로 남습니다.

### 제약

- **Date 타입 context 변수는 매핑되지 않습니다.** 올바른 파싱은 Studio 에 선언된 변수별 포맷 패턴에 달려 있는데 routine 은 그 패턴을 알 수 없습니다. 문자열로 선언해 Job 안에서 파싱하거나, 서버의 실행 인자 경로에 위임하세요.
- 적재 유형은 `PARAM_LOAD_TYPE` 이 아니라 `LOAD_TYPE` 으로 주입되므로 이 routine 의 대상이 아닙니다. 필요하면 `System.getenv("LOAD_TYPE")` 을 직접 읽습니다.
- 파라미터 키와 context 변수명이 이름 자체로 다르면 어느 경로로도 값이 전달되지 않습니다. routine 이 흡수해 주는 것은 대소문자 차이뿐입니다.

> **참고** 서버가 실행 인자(`--context_param`)로도 값을 보내지만 routine 호출은 계속 두는 것이 좋습니다. 시크릿은 환경변수 경로로만 전달되고, 실행 인자를 지원하지 않는 이전 서버에서도 routine 은 같은 방식으로 동작하기 때문입니다. 실행 인자 주입은 서버 설정 `execution.context-param.enabled`(기본 `true`)로 끌 수 있으며, 꺼도 환경변수 경로는 유지되므로 routine 을 쓰는 Job 은 영향을 받지 않습니다.

## 5.5 NeoFlowJobApi 로 서버 값 조회

`NeoFlowJobApi`(패키지 `com.neoflow.jobapi`)는 Job 안에서 NeoFlow Server 의 실행 컨텍스트를 조회하는 정적 메서드 모음입니다. NeoFlow 컴포넌트와 함께 배포되는 `neoflow-job-api-1.0.0.jar` 에 들어 있고, `tNeoFlowPre` 가 Job 에 `import com.neoflow.jobapi.*;` 를 자동으로 넣으므로 `tNeoFlowPre` 가 있는 Job 에서는 클래스 이름만으로 호출할 수 있습니다.

| 메서드 | 반환 타입 | 설명 |
|---|---|---|
| `getParameter(String key)` | `Object` | 이번 실행의 파라미터 값 하나. 없으면 null. 호출할 때마다 서버에 한 번 조회합니다. |
| `pullRuntimeContext()` | `RuntimeContext` | BizDate 구간·적재 유형·실행 ID·파라미터 전체 (tNeoFlowPre 가 내부에서 사용) |
| `commitBizDate(LocalDateTime end)` | `CommitResult` | BizDate 워터마크 커밋 (tNeoFlowPost 가 내부에서 사용) |
| `commitNow()` | `CommitResult` | 현재 시각으로 워터마크 커밋 |
| `getEnv()` | `Map<String, String>` | 연결 설정 확인용. 토큰 값은 ********로 가려서 돌려줍니다. |

### 사용 예 — DB 접속 정보를 공통 설정에서 받기

컴포넌트의 입력 칸에 Java 식으로 적습니다. 값은 문자열로 변환해 씁니다.

```
Host     : (String)NeoFlowJobApi.getParameter("MYSQL_HOST")
Port     : (String)NeoFlowJobApi.getParameter("MYSQL_PORT")
Password : (String)NeoFlowJobApi.getParameter("MYSQL_PASSWORD")
```

공통 설정 값은 문자열로 돌아옵니다. 실행할 때 입력한 값에 숫자를 넣으면 숫자형(`Long`)으로 돌아오므로, 숫자가 올 수 있는 파라미터는 `String.valueOf(NeoFlowJobApi.getParameter("키"))` 로 변환하면 안전합니다.

### 값의 출처

- NeoFlow Server 에서 실행될 때 — 그 실행의 파라미터를 서버가 발급한 실행 토큰으로 조회합니다. 값은 실행 시 입력 > Job 파라미터 기본값 > 공통 설정(활성 프로파일) 순으로 정해지며, 비밀 속성도 평문으로 받습니다.
- Studio 테스트 실행 — 같은 실행에서 `tNeoFlowPre` 가 연 개발용 세션으로 서버 기본값(Job 파라미터 기본값 > 공통 설정)을 조회합니다. 개발·검증 서버에서만 동작하며, 운영(prod) 서버에서는 세션이 열리지 않아 호출이 예외로 끝나고 Job 이 실패합니다(3.6).

> **참고** `NeoFlowContext` 와의 차이 — `NeoFlowContext` 는 서버가 넣어 준 `PARAM_*` 환경변수를 context 변수에 옮길 뿐 서버를 호출하지 않으므로, 로컬 실행에서는 Context 탭 기본값이 그대로 남습니다. `NeoFlowJobApi.getParameter()` 는 context 변수 없이 서버에서 직접 값을 받지만, 서버 연결이 필요하고 `tNeoFlowPre` 가 있는 Job 에서만 쓸 수 있습니다.

> **주의** `getParameter()` 로 받은 값을 `System.out.println` 등으로 출력하지 마세요. 비밀 속성도 평문으로 돌아오며, Job 의 표준출력은 실행 로그에 그대로 남습니다.

# 6. Sync to NeoFlow Server

작성한 Job 을 빌드하여 NeoFlow 서버로 메타데이터 동기화 + 빌드 산출물 업로드까지 한 번에 수행합니다.

> **주의** 전제 조건 — 유효한 License(2 장)와 NeoFlow Server 프로파일(3 장)이 등록되어 있어야 합니다. License 가 유효하지 않으면 메뉴 실행 시 에러 다이얼로그가 뜨고 마법사가 열리지 않습니다.

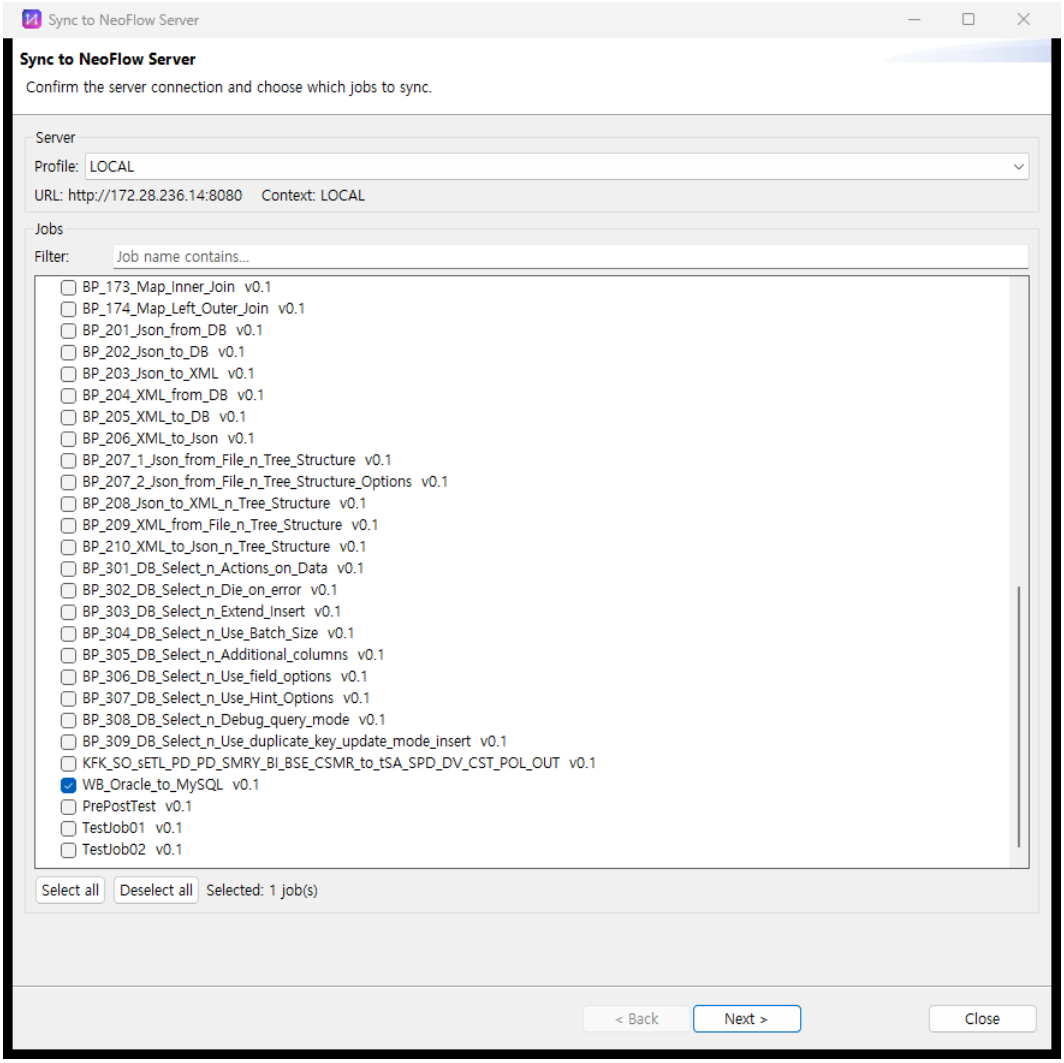
## 6.1 실행

Repository 뷰에서 Job(또는 Job 이 든 폴더)을 하나 이상 선택한 뒤 우클릭 → [Sync to NeoFlow Server...] 를 실행합니다.

## 6.2 마법사 단계

### Page 1 — 서버/잡 선택

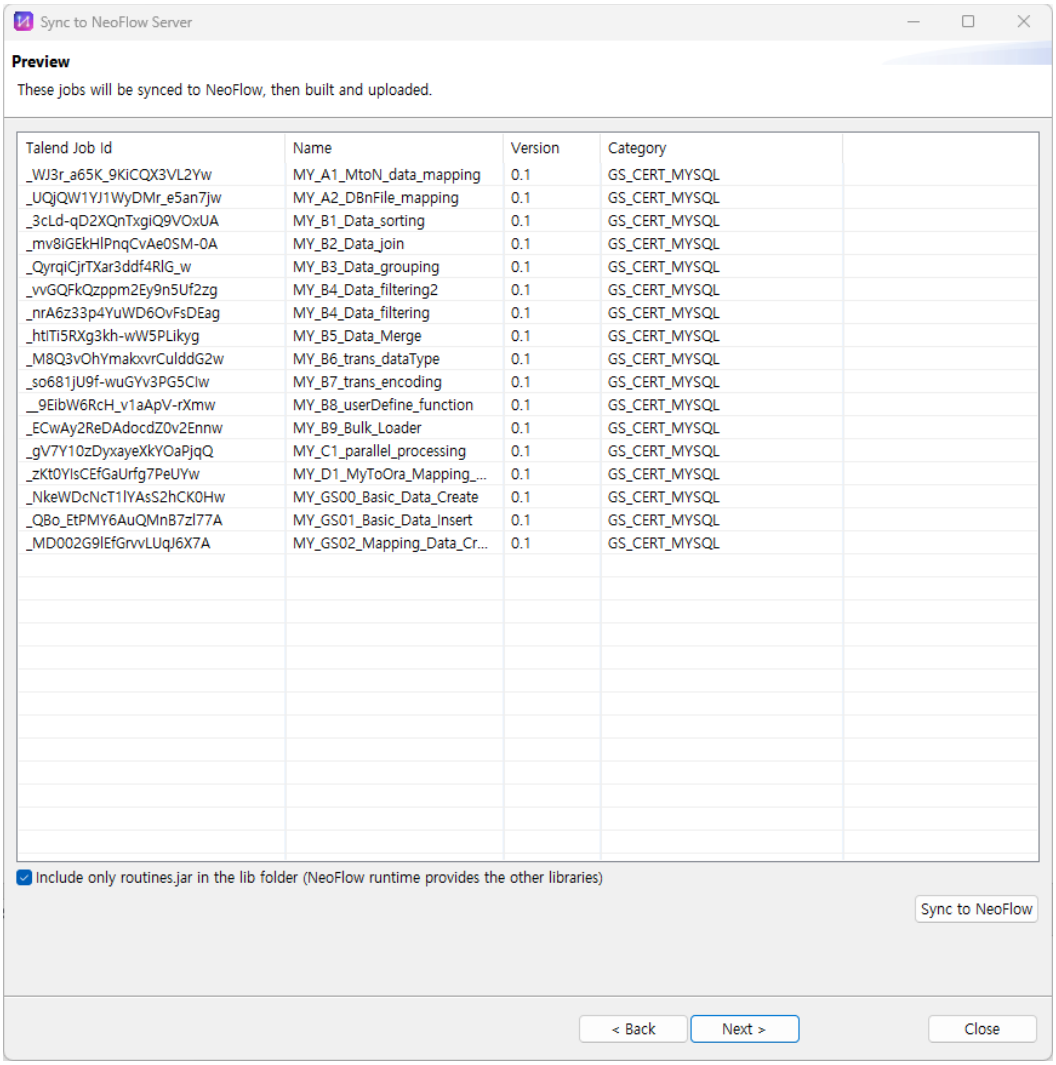
- Profile — 사용할 서버 프로파일 선택(아래에 URL/Context 표시). 기본 프로파일이 미리 선택됩니다.
- Filter — Job 이름으로 목록을 필터링합니다.
- Job 목록(체크 트리)에서 동기화할 Job 을 체크합니다. 각 행은 잡이름 v 버전 으로 표시됩니다.
- Select all / Deselect all 버튼과 하단 `Selected: N job(s)` 표시를 제공합니다.
- 진행 조건 — Job 1 개 이상 선택 + 프로파일에 Base URL 과 API token 이 있어야 합니다.




*Sync 마법사 Page 1 — Profile 선택 + Job 체크(Selected: N job(s))*

## Page 2 — Preview

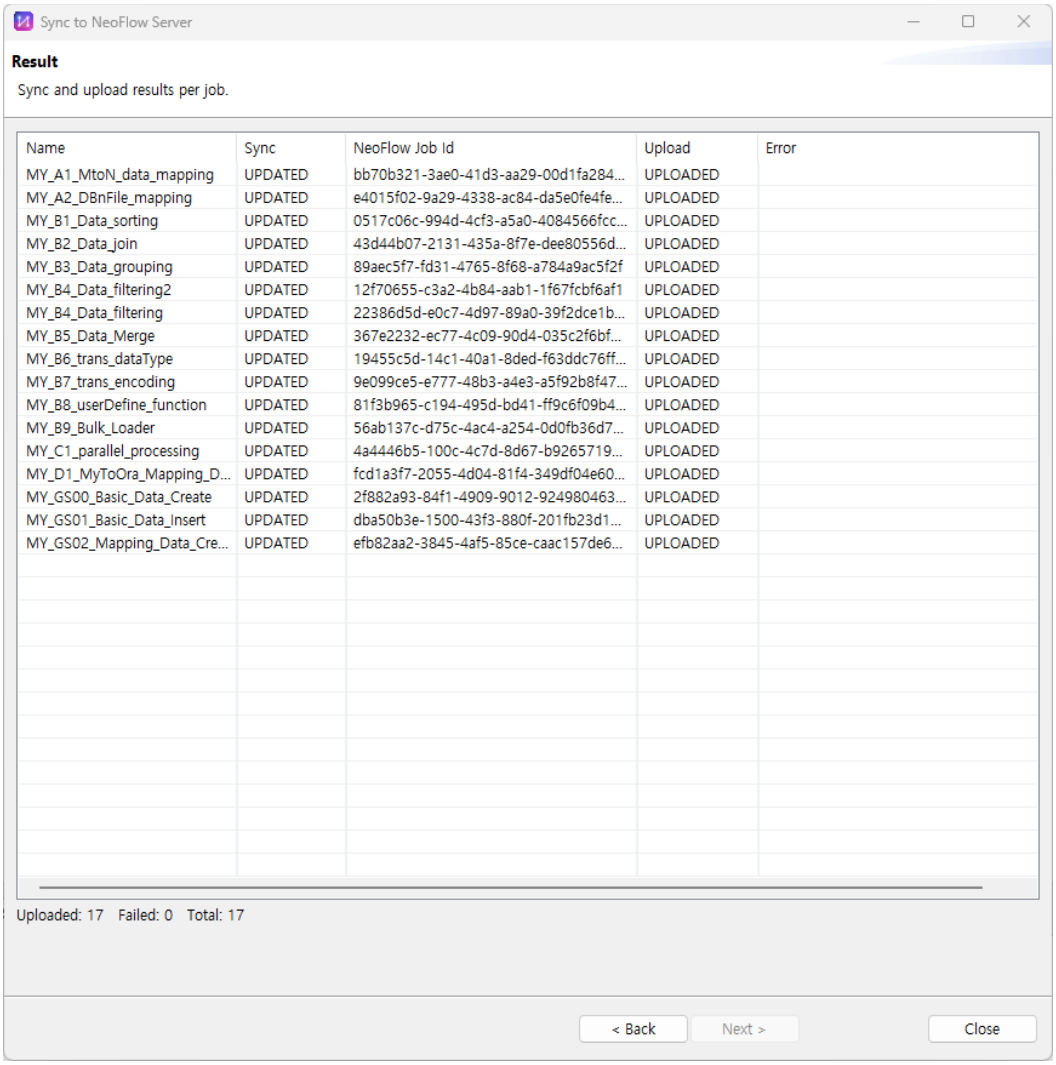
- 선택한 Job 의 메타데이터 표시 — `Talend Job Id`, `Name`, `Version`, `Category`.
- Include only routines.jar in the lib folder (기본 체크) — 체크 시 산출물 `lib/`에 `routines.jar` 만 넣고, 나머지 라이브러리는 실행할 때 서버의 공유 라이브러리(NEO_LIBS)에서 받습니다. 서버에 없는 라이브러리를 쓰는 Job 은 관리자가 웹 콘솔의 「공유 라이브러리 (NEO_LIBS)」에 올리거나, 이 옵션을 해제하고 Sync 하세요(해제하면 `lib/`의 jar 가 업로드 때 공유 라이브러리에 등록됩니다).
- Sync to NeoFlow 버튼 — 동기화 + 빌드 + 업로드 과정을 시작합니다.




◢ *Sync 마법사 Page 2 — 메타데이터 확인 + routines.jar 옵션 + Sync to NeoFlow*

## Page 3 — Result

- Job 별 결과 표시 — `Name`, `Sync`, `NeoFlow Job Id`, `Upload`, `Error`.
- 하단 요약 — `Uploaded: N  Failed: N  Total: N`.
- 행 우클릭 — Copy error(에러 복사), Copy row(행 전체 TSV 복사).




◢ *Sync 마법사 Page 3 — Job 별 결과(UPDATED/UPLOADED) + 요약*

> **참고** 이 다이얼로그는 비모달입니다. 빌드가 오래 걸려도 Studio 의 다른 작업이 가능하며, Close 버튼으로 닫습니다.

## 6.3 내부 동작 요약

1. 메타데이터 동기화 — 선택한 Job 정보를 서버에 등록하고 NeoFlow Job ID 를 발급받습니다.
2. context 주입 — 빌드 전, 프로파일의 `Build context` 환경에 `NEOFLOW_BASE_URL`/`NEOFLOW_API_TOKEN`/`NEOFLOW_JOB_ID` 를 기록하고, 이 환경을 Job 의 기본 context 로 설정합니다.
3. 빌드 — Job 을 standalone POJO zip 으로 빌드합니다(원본 `.item` 소스 포함, source code 제외).
4. 라이브러리 정리 — 위 체크박스가 켜져 있으면 `lib/`에서 `routines.jar` 만 남기고 제거합니다.
5. 업로드 — 빌드된 zip 을 서버에 업로드합니다. 서버는 zip 안 context 파일의 `NEOFLOW_API_TOKEN` 값(및 `nf_` 로 시작하는 토큰 값)을 비워서 저장하므로, 서버에 저장되거나 다운로드되는 패키지에는 API 토큰이 남지 않습니다. 서버에서 실행할 때는 서버가 넣어 주는 값을 쓰므로 실행에는 영향이 없습니다.

> **주의** Studio 작업 공간의 Job(`.item`)과 로컬에 만든 빌드 zip 에는 API 토큰이 평문으로 남아 있습니다. Job 파일이나 로컬 빌드 zip 을 다른 사람에게 전달할 때는 context 의 `NEOFLOW_API_TOKEN` 값을 지우고, 이미 전달했다면 [설정] → [API 토큰]에서 해당 토큰을 폐기하십시오.

> **참고** 처음 Sync 한 Job 은 NeoFlow Server 에 비활성 상태로 등록됩니다. 실행하거나 스케줄에 올리려면 웹 콘솔 Job 목록에서 [활성화] 하세요(여러 개는 선택한 뒤 [선택 활성화], ADMIN·매니저). 이미 등록된 Job 을 다시 Sync 하면 상태는 그대로 유지됩니다.

> **참고** Job 안에 `tNeoFlowPre`/`tNeoFlowPost` 가 있으면 context 주입이 수행되고, `tNeoFlowPre` 의 `BASE_URL`/`API_TOKEN`/`JOB_ID` 파라미터가 `context.NEOFLOW_*` 로 자동 연결됩니다. `tNeoFlowPost` 에는 이 파라미터가 없으며, 서버 실행 때 서버가 주입하는 실행 토큰을 씁니다(3.6). NeoFlow 컴포넌트가 없는 Job 은 context 가 수정되지 않습니다.

## 6.4 Sync 전후의 Job 처리

Sync 와 Build Job 의 Deploy(7 장)는 Job 을 서버에 올리기 전후에 다음 두 가지를 자동으로 처리합니다.

### 열린 Job 편집기 저장·닫기

[Sync to NeoFlow] 를 누르면 선택한 Job 가운데 편집기에 열려 있는 Job 을 먼저 저장하고 닫은 뒤 진행합니다. 편집기에 열린 Job 은 저장된 Job 과 별개의 편집 상태를 가지므로, 열어 둔 채로 서버 연결 정보를 기록하면 다음 저장 때 덮어써지기 때문입니다. 저장하지 않은 변경이 있으면 저장 여부를 묻는 창이 뜹니다.

| 상황 | 메시지 (화면 표시 그대로) |
|---|---|
| Sync 중 저장 창에서 취소 | `Sync cancelled: save or close the open job(s) first.` |
| Deploy 중 저장 창에서 취소 | `Deploy cancelled: save or close the open job(s) first.` |
| Sync 진행 중 취소 | `Sync cancelled.` |
| 서버 오류 등으로 Sync 중단 | `Sync aborted: <오류 내용>` |

### Job 사본의 NeoFlow 연결 해제

Repository 에서 [Duplicate] 하거나 [Copy] → [Paste] 로 만든 Job 사본은 아직 서버에 등록되지 않은 새 Job 입니다. 그래서 사본의 context 변수 `NEOFLOW_BASE_URL`·`NEOFLOW_API_TOKEN`·`NEOFLOW_TOKEN`·`NEOFLOW_JOB_ID` 값을 비워, 원본이 등록된 서버 Job 과 분리합니다. 변수 자체는 남겨 두므로 `tNeoFlowPre` 의 설정은 그대로 유지됩니다.

> **참고** 사본을 서버에 올리려면 사본을 Sync 하세요. Sync 가 사본 자신의 NeoFlow Job ID 로 값을 다시 채웁니다.

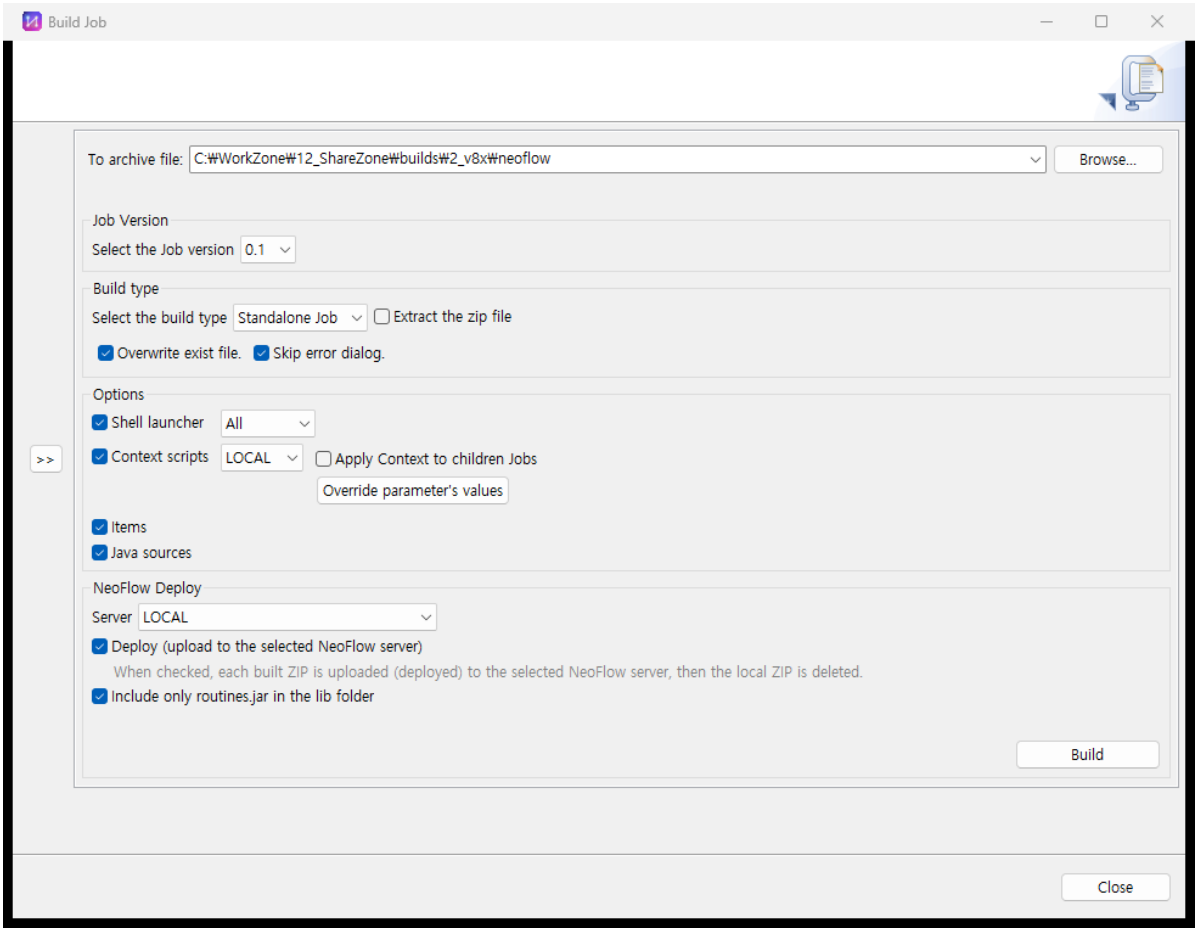
# 7. Build Job (NeoFlow Deploy)

Job 을 로컬 zip 으로 빌드하면서, 옵션으로 NeoFlow 서버에 배포(Deploy)까지 한 번에 할 수 있습니다. Sync 마법사가 "서버 등록 + 업로드"에 특화돼 있다면, Build Job 은 표준 빌드 산출물을 만들면서 같은 배포를 곁들이는 방식입니다.

> **주의** 전제 조건 — Sync 와 동일하게 유효한 License 와 NeoFlow Server 프로파일이 필요합니다. License 가 유효하지 않으면 Build Job 메뉴 실행 시 에러 다이얼로그가 뜹니다.

## 7.1 실행

Repository 에서 Job(또는 폴더)을 우클릭 → [Build Job] 을 실행합니다.




*▲ Build Job 다이얼로그 — 표준 export 옵션 + NeoFlow Deploy 섹션*

## 7.2 주요 항목

- To archive file — 빌드 산출물(zip)을 저장할 경로.
- Job Version / Build type — 버전, 빌드 타입(Standalone Job 등), Overwrite/Skip error 옵션.
- Options — Shell launcher, Context scripts(환경 선택), Items(원본 `.item` 포함), Java sources 등 표준 export 옵션.

### NeoFlow Deploy (NeoFlow 전용)

- Server — 배포할 NeoFlow 서버 프로파일(3 장에서 등록).

- Deploy (upload to the selected NeoFlow server) — 기본 체크. 체크 시 빌드된 zip 을 선택한 NeoFlow 서버로 업로드(배포)합니다. 로컬 zip 만 만들려면 해제합니다.
- Include only routines.jar in the lib folder — 기본 체크. 체크 시 `lib/`에 `routines.jar` 만 넣고 나머지는 실행할 때 서버의 공유 라이브러리(NEO_LIBS)에서 받습니다(Sync 의 동일 옵션과 같음, 6.2).

> **주의** Deploy 안내 문구: "When checked, each built ZIP is uploaded (deployed) to the selected NeoFlow server, then the local ZIP is deleted." — 배포 후 로컬 zip 은 삭제됩니다(서버 업로드가 목적이므로). 로컬 보관이 필요하면 Deploy 를 해제하고 빌드만 하세요.

> **참고** Deploy 가 켜진 Build Job 은 내부적으로 Sync 와 동일한 동작(메타데이터 동기화 + `context.NEOFLOW_*` 주입 + 업로드)을 수행합니다(6.3 참고).

## 7.3 배포 실행과 결과

[Deploy] 를 켠 채 [Build] 를 누르면 Sync 와 같은 순서(열린 편집기 정리 → 메타데이터 동기화 → context 주입 → 빌드 → 업로드)로 처리한 뒤, `NeoFlow Deploy Result` 창에 Job 별 결과를 보여 줍니다.

- 결과 창의 열 — `Name`, `Sync`, `Upload`, `Error`.
- 하단 요약 — `Uploaded: N  Failed: N  Total: N`.

Build Job 창은 비모달 창입니다. 빌드가 진행되는 동안에도 Studio 의 다른 작업을 할 수 있으며, [Close] 버튼이 계속 활성화되어 있어 진행 중에 중단을 요청할 수 있습니다.

배포 중 문제가 있으면 제목이 `NeoFlow Deploy` 인 창에 다음 문구가 표시됩니다.

| 상황 | 메시지 (화면 표시 그대로) |
|---|---|
| 서버 프로파일 없음 | `No NeoFlow server profile is configured. Set one in Preferences > NeoFlow Server, or uncheck Deploy to build locally only.` |
| 선택한 Job 없음 | `No jobs selected to deploy.` |
| 편집기 저장 창에서 취소 | `Deploy cancelled: save or close the open job(s) first.` |
| 진행 중 취소 | `Deploy cancelled.` |
| 서버 오류 등으로 중단 | `Deploy aborted: <오류 내용>` |

Deploy 를 해제하고 빌드하다 실패하면 `N job(s) did not build:` 로 시작하는 창에 실패한 Job 과 `See the Error Log view for the full stack trace.` 가 표시됩니다. Error Log 뷰에서 원인을 확인하세요.

> **참고** 메시지 속 `Preferences > NeoFlow Server` 는 화면 문구 그대로이며, 실제 위치는 [Window] → [Preferences] → [NeoFlow] → [NeoFlow Server] 입니다(3.2).

# 8. 설치와 제거

NeoFlow Studio 는 설치 프로그램(installer) 없이 배포 압축 파일을 풀어 사용합니다. 이 장은 PC 요구 사항, 압축 해제, JDK 지정, 첫 실행과 버전 확인, 제거 방법을 다룹니다. 화면 구성과 기본 조작은 «NeoFlow Studio 시작하기 가이드»를 참고하세요.

## 8.1 시스템 요구 사항

| 항목 | 요구 사항 | 비고 |
|---|---|---|
| OS | Windows 10 / 11 (64bit) | 32bit Windows 는 지원하지 않습니다. |
| Java | JDK 11 (64bit) | Studio 는 JDK 11 로 실행됩니다(ini 의 -Dosgi.requiredJavaVersion=11). |
| 메모리 | 8GB 이상 | Studio 최대 힙이 4GB(-Xmx4096m)로 설정되어 있습니다. |
| 압축 해제 경로 | 짧은 영문 경로 | 예: C:\NEO_BD-V1.0.0 |
| 네트워크 | NeoFlow Server 접속 | Sync·Build Job(Deploy) 때 서버 Base URL 로 접속합니다. |

## 8.2 압축 해제

1. 배포받은 NeoFlow Studio 압축 파일을 C 드라이브 바로 아래 같은 짧은 영문 경로에 풉니다(예: `C:\NEO_BD-V1.0.0`).
2. 압축을 푼 폴더에 아래 항목이 있는지 확인합니다.

| 항목 | 설명 |
|---|---|
| `NEO_BD-win-x86_64.exe` | Studio 실행 파일 (Windows 64bit) |
| `NEO_BD-win-x86_64.ini` | JDK 경로·메모리 등 실행 옵션 (8.3) |
| `workspace` | 프로젝트·Job 이 저장되는 작업 폴더. 처음 실행할 때 만들어집니다 (백업 대상) |
| `license.bin` | 라이선스 등록 후 생성되는 파일 (2 장) |
| `plugins, features, configuration` | Studio 본체 — 수정하지 마세요 |
| `custom-components` | 컴포넌트 업데이트용 폴더 — 수정하지 마세요 |

> **주의** 압축 해제 경로에 한글·공백·특수문자가 들어가거나 경로가 길면 실행이나 Job 빌드가 실패할 수 있습니다. 바탕 화면·사용자 폴더 대신 드라이브 루트 아래 짧은 영문 경로를 쓰세요(Windows 경로 길이 제한 260 자).

## 8.3 JDK 지정 (NEO_BD-win-x86_64.ini)

Studio 가 쓸 JDK 는 실행 파일과 같은 폴더의 `NEO_BD-win-x86_64.ini` 첫 두 줄에 지정합니다. 첫 줄은 `-vm`, 둘째 줄은 `javaw.exe` 의 전체 경로입니다. 배포 압축 파일의 ini 에는 이 두 줄이 없을 수 있으므로, 없으면 파일 맨 위(`-vmargs` 보다 위)에 추가합니다. 지정한 예는 다음과 같습니다.

```
-vm
```

```
C:\Program Files\Java\jdk-11\bin\javaw.exe
-vmargs
-Xms1536m
-Xmx4096m
```

둘째 줄에는 실제로 설치한 JDK 11 의 경로를 씁니다.

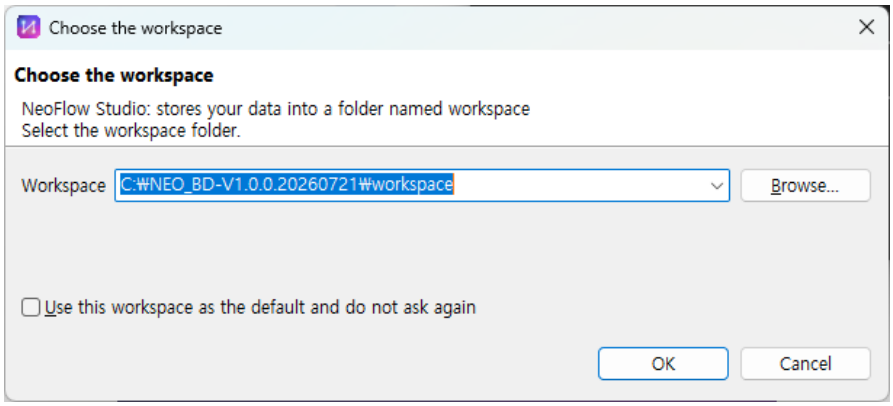
1. JDK 11(64bit)을 설치하고, 설치 폴더 안의 `bin\javaw.exe` 전체 경로를 확인합니다.
2. Studio 를 종료한 상태에서 `NEO_BD-win-x86_64.ini` 를 메모장으로 엽니다.
3. 맨 위에 `-vm` 줄과 확인한 `javaw.exe` 전체 경로 줄을 넣습니다(이미 있으면 둘째 줄의 경로만 고칩니다). 경로에 공백이 있어도 따옴표로 감싸지 않습니다.
4. `-vm` 과 경로는 각각 한 줄씩 두고, 두 줄 모두 `-vmargs` 보다 위에 있어야 합니다. 나머지 줄은 수정하지 않습니다.
5. 저장한 뒤 Studio 를 실행합니다.

> **주의** `-vm` 경로가 틀리면 Studio 를 실행할 때 Java 가상 머신을 찾을 수 없다는 오류 창이 뜨고 Studio 가 시작되지 않습니다. 이 경우 ini 둘째 줄의 경로와 파일 이름(`javaw.exe`)을 다시 확인하세요.

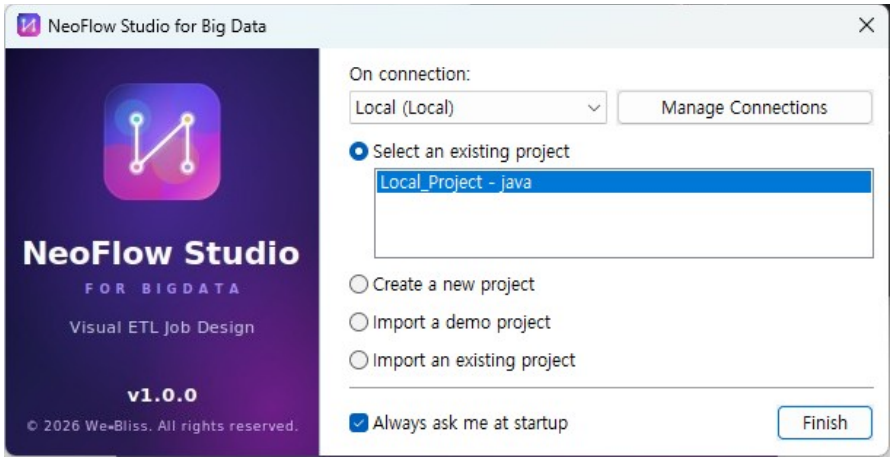
> **참고** `-vmargs` 아래의 `-Xms1536m`·`-Xmx4096m` 이 Studio 의 시작·최대 메모리입니다. 요구 사항(8GB 이상)을 만족하는 PC 에서는 그대로 둡니다.

## 8.4 첫 실행과 버전 확인

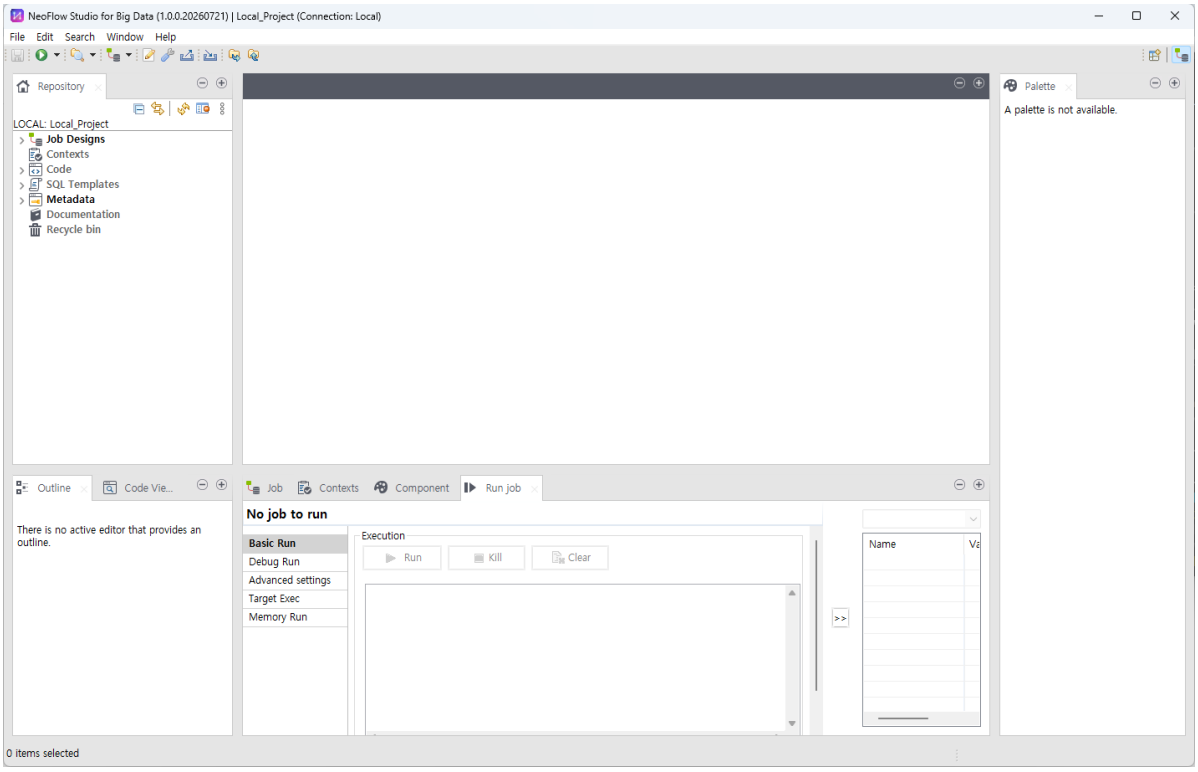
1. 압축을 푼 폴더에서 `NEO_BD-win-x86_64.exe` 를 더블 클릭합니다.
2. 워크스페이스 선택 창이 나타나면 기본값(설치 폴더 아래 `workspace`)을 확인하고 [OK] 를 누릅니다.
3. 프로젝트 선택 창에서 `Select an existing project` 목록의 프로젝트를 고르거나, `Create a new project` 로 새 프로젝트를 만든 뒤 [Finish] 를 누릅니다.
4. Studio 메인 화면이 열리면 창 제목 표시줄에 `NeoFlow Studio for Big Data` 와 버전(1.0.0 으로 시작)이 표시되는지 확인합니다. [Help] → [About NeoFlow Studio] 창에서도 제품 정보를 확인할 수 있습니다.
5. 이어서 2 장의 License 등록을 진행합니다.




▲ *워크스페이스 선택 창*




▲ *프로젝트 선택 창 — Select an existing project / Create a new project*




▲ *Studio 메인 화면 — 창 제목에 NeoFlow Studio for Big Data 와 버전 v1.0.0 표시*

## 8.5 제거

설치 프로그램 없이 압축을 풀어 쓰므로 제거도 폴더를 지우는 방식입니다.

1. 보관할 Job 이 있으면 먼저 백업합니다. Studio 를 종료하고 `workspace` 폴더를 다른 위치로 복사하거나, Repository 에서 Job 을 [Export items] 로 내보냅니다.
2. Studio 가 종료되었는지 확인한 뒤 압축을 풀었던 설치 폴더 전체를 삭제합니다. `license.bin` 과 `workspace` 도 이 폴더 안에 있으므로 함께 지워집니다.
3. Studio 용으로만 설치한 JDK 가 더 필요 없으면 Windows [설정] → [앱] 에서 제거합니다.

> **참고** 다른 PC 로 옮길 때는 백업한 `workspace` 를 새 설치 폴더에 두고 첫 실행에서 그 워크스페이스를 선택합니다. 특정 PC 의 MAC 주소로 발급된 라이선스는 새 PC 에서 유효하지 않으므로 다시 발급받아야 합니다 (2.2).

# 9. 부록

## 9.1 빠른 점검 체크리스트

Job 을 서버에 올리기 전에 다음을 순서대로 확인하세요.

- 설치 — `NEO_BD-win-x86_64.ini` 맨 위의 `-vm` 경로가 JDK 11(64bit) `javaw.exe` 를 가리키고, 창 제목에 `NeoFlow Studio for Big Data` 와 버전(1.0.0 으로 시작)이 표시되는지 확인.
- License — [Help] → [License Registration]에서 `License key is valid.` 확인.
- API token — NeoFlow Server 관리 콘솔 [설정] → [API 토큰]에서 토큰 발급 후 값 복사(발급 직후 1 회만 표시).
- Server 프로파일 — [Window] → [Preferences] → [NeoFlow] → [NeoFlow Server]에서 Base URL/API token 입력, Test Connection 성공, Set as default 지정.
- Job 설계 — BizDate 사용 시 `tPrejob` 뒤에 `tNeoFlowPre`, `tPostjob` 뒤에 `tNeoFlowPost` 를 `OnComponentOk` 로 연결. 병렬 처리 시 `tNeoFlowThread`.
- Sync — Repository 에서 Job 우클릭 → [Sync to NeoFlow Server...] → Job 선택 → Sync → Result 에서 `Uploaded` 확인.

## 9.2 자주 묻는 질문

**Q. Sync/Build Job 메뉴를 눌렀더니 에러 다이얼로그만 뜹니다.**

A. License 가 없거나 만료/불일치 상태입니다. 2 장을 참고해 라이선스를 다시 등록하세요.

**Q. Test Connection 이 실패하거나 Sync 에서 인증 오류가 납니다.**

A. Test Connection 창에 표시된 문구로 원인을 구분합니다. `Invalid or expired API token` 은 토큰이 만료·폐기됐거나 잘못 붙여넣은 경우이고(토큰을 발급한 계정이 비활성이어도 같습니다), `Token lacks the ADMIN/JOB_MANAGER role` 은 토큰을 발급한 계정이 ADMIN·매니저가 아닌 경우입니다. `Cannot reach server:` 뒤에 `PKIX path building failed` 가 나오면 서버 인증서를 Studio 가 쓰는 JDK 의 `cacerts` 에 등록하고 Studio 를 다시 시작하세요(3.5). 토큰 문제라면 NeoFlow Server 관리 콘솔 [설정] → [API 토큰]에서 상태와 만료일을 확인하고, 필요하면 새 토큰을 발급해 프로파일에 다시 입력하세요(3.1).

**Q. 발급받은 토큰을 복사하지 못하고 모달을 닫았습니다.**

A. 평문 토큰은 다시 볼 수 없습니다. 목록에서 해당 토큰을 [토큰 폐기]로 폐기하고 새로 발급하세요.

**Q. tNeoFlowPre 를 넣었는데 nf_startStr/nf_endStr 가 비어 있습니다(null).**

A. 서버 바인딩이 없는 상태입니다. Studio 단독 테스트라면 Sync 를 한 번 수행해 `context.NEOFLOW_*`가 채워졌는지 확인하세요. 연결한 서버가 운영(prod) 프로파일이면 로컬 테스트 실행에서 개발 토큰이 거부되어 값이 채워지지 않습니다(3.6). `tNeoFlowPre` 는 never-fail 이라 바인딩이 없으면 값을 채우지 않고 통과합니다.

**Q. Studio 에서 Job 을 실행하니 「studio context unavailable」 로그가 남고 DB 연결 단계에서 실패합니다.**

A. 연결한 NeoFlow Server 가 운영(prod) 프로파일이면 로컬 테스트 실행용 개발 토큰 발급이 403 으로 거부됩니다. 그러면 `tNeoFlowPre` 가 구간을 받지 못하고, 접속 정보를 `NeoFlowJobApi.getParameter()` 로 받는 컴포넌트도 값을 조회하지 못해 실패합니다. 운영 서버에서는 Sync·활성화한 뒤 웹 콘솔에서 실행하고, 로컬 테스트는 개발·검증 서버 프로파일로 하세요(3.6).

**Q. tNeoFlowPost 가 커밋을 건너뜁니다.**

**A.** 같은 Job 에 `tNeoFlowPre` 가 먼저 실행되어 `nf_endDate` 를 채웠는지 확인하세요. 배치가 어긋나 `tNeoFlowPost` 가 먼저 실행되는 경우도 원인이 됩니다 — `tNeoFlowPre` 는 `tPrejob` 뒤, `tNeoFlowPost` 는 `tPostjob` 뒤에 두어야 합니다(4 장 배치 규칙). 커밋은 됐는데 건너뛴 경우에는 `SKIP_REASON` 을 확인하세요. `NON_MONOTONIC` 은 이번 종료 시각이 이미 커밋된 워터마크보다 늦지 않은 경우(같은 Job 의 실행이 겹쳐 다른 실행이 먼저 커밋했거나, 워터마크를 수동으로 앞당겨 둔 경우 등), `BACKFILL` 은 소급 적재 실행이라 워터마크를 옮기지 않은 경우입니다.

**Q. NeoFlowContext routine 을 따로 등록해야 하나요?**

**A.** 아닙니다. `NeoFlowContext` 는 시스템 routine 으로 Studio 에 포함되어 있습니다([Code] → [Routines] → [system]). 같은 이름의 사용자 routine 은 만들지 마세요(5.2).

**Q. Studio 를 실행하면 Java 를 찾을 수 없다는 오류 창이 뜨고 시작되지 않습니다.**

**A.** `NEO_BD-win-x86_64.ini` 둘째 줄의 JDK 경로가 실제 설치 위치와 다릅니다. JDK 11(64bit)의 `bin\javaw.exe` 전체 경로로 고친 뒤 다시 실행하세요(8.3).

**Q. tNeoFlowThread 에서 일부 스레드가 실패해도 Job 이 성공으로 끝납니다.**

**A.** `Die on error` 가 해제되어 있기 때문입니다. 실패 시 Job 을 중단하려면 체크하세요.